

# Introduction

Voluntary task switching requires individuals to coordinate their actions across competing tasks. Even under identical experimental conditions, individuals differ considerably in how they organize their responses. The free concurrent dual-tasking (FCDT) paradigm provides an experimentally controlled setting for investigating these differences while allowing participants to develop their own response strategies. Two task streams remain concurrently visible, and participants decide which task to perform on each response. Some individuals predominantly perform long sequences of one task before switching, whereas others frequently interleave the two tasks. Still others exhibit response grouping, characterized by closely spaced responses to both tasks following a longer processing interval. The general aim of our work is to understand why different observers adopt these different response strategies and which subjective constraints make their choices sensible courses of action.

Our modeling approach follows the rational analysis program as outlined in the project proposal (Brüning et al., 2025). Rational analysis assumes that an observer’s choices can be understood as optimal with respect to a particular representation of the decision problem and its subjective costs and benefits. For the FCDT paradigm, relevant objectives include accurate and exhaustive task performance, avoiding costly task switches, and maintaining a balance between the two tasks. Given a formal specification of these objectives, dynamic programming provides algorithmic means to derive an optimal decision policy. Conversely, observed response sequences can inform the subjective cost parameters under which such a policy would be optimal. This inverse dynamic programming (Drugowitsch et al., 2012; Kochenderfer et al., 2022; Ramachandran & Amir, 2007) perspective connects individual response strategies to quantitative hypotheses about their determinants. To enable scientific inference about these hypothesized determinants, we adopt the agentic behavioral modeling framework of Ostwald et al. (2026). This framework combines a formal representation of the experimental task, an agent model of the hypothesized cognitive mechanisms, and a probabilistic data model linking the agent’s decisions to observed behavior. The resulting likelihood supports parameter estimation and model comparison, allowing competing accounts of response strategy formation to be evaluated against empirical data.

In Brüning et al. (2025), we provided a preliminary proof of principle through generative simulations of a simplified FCDT paradigm involving letter and digit categorization. The agent’s reward function combined correct-response rewards with costs for switching tasks and accumulating task imbalance. Varying the relative strength of these costs produced qualitatively different response sequences while preserving accurate classification. High switching costs favored extended task blocks, whereas lower switching costs yielded increasingly frequent task alternation, including choice patterns associated with response grouping. These simulations demonstrated that differences in response organization could arise from a common optimization architecture with different subjective cost parameters.

However, while these simulations provided an encouraging proof of principle for the rational analysis of FCDT response strategies, their scope remained limited in both theoretical and empirical respects.

The finite-trial task formulation was solved through a discounted infinite-horizon approximation using policy iteration, yielding stationary policies that did not account for the approaching end of an experimental block. This approximation also introduced a preference for earlier over later rewards without an independently established psychological justification in the present setting. Computational demands further restricted the demonstration to four letters and four digits, simplifying the choice environment relative to the experimental paradigm. The resulting account thus established the generative plausibility of the proposed mechanisms, while their explanatory value for individual participants remained unexamined, as the approach was neither applied to empirical response sequences nor used to infer participants’ subjective cost parameters.

The present report takes a first step toward addressing these limitations by applying our rational analysis approach to empirical FCDT data from Brüning et al. (2020). We extend the task formulation to accommodate the richer experimental setting and derive optimal policies by backward induction for a finite-horizon MDP without reward discounting. This allows decisions to reflect the remaining response horizon while retaining a computationally tractable account of task performance. Embedding these policies within the agentic behavioral modeling framework makes their explanatory value amenable to statistical evaluation and enables estimation of individual subjective cost parameters from observed response sequences.

The present report addresses the organization of response choices as a first step toward a temporal account of FCDT performance. In future work on this project, we will extend this analysis within a semi-Markov framework, deriving optimal policies jointly over task responses and waiting times to capture both what participants choose and when they respond. This extension will relate response strategies to the experimental deadline and the objective of maximizing correct-response throughput, while accounting for the contribution of stimulus preview and preparation of the alternative task to multitasking efficiency (Brüning et al., 2020). Doing so requires a more explicit representation of processing time, cognitive effort, and concurrent task preparation within the agent’s subjective costs and benefits. The current focus on response choices establishes the theoretical and statistical foundation for this subsequent analysis of the timing and efficiency of task performance.

The [Experimental data](#) section reviews the FCDT data reported by Brüning et al. (2020) and describes the PTD-RMC condition analyzed here. The [Agentic behavioral model](#) section specifies the task, agent, and data models used in our analysis, while the underlying MDP theory is provided in [Supplement S1](#). We then describe the [model estimation and evaluation](#) procedures. The [Preliminary results](#) section reports the parameter estimates and model-comparison results. Finally, the [Outlook](#) summarizes the next steps of the project.

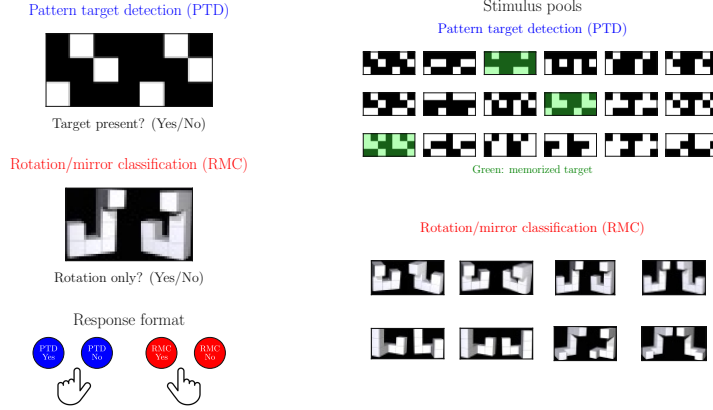
## Experimental data

For this report, we developed our approach for a subset of the data reported in Br uning et al. (2020). Using the free concurrent dual-tasking paradigm, the authors investigated whether competition for perceptual-cognitive and response-related resources affects how individuals organize their responses to two concurrent tasks and how efficiently they perform them. Across two experiments, they examined individual preferences for ‘blocking’, ‘switching’, and ‘response grouping’ under task combinations that differed in their processing codes and response modalities. The original data and stimulus materials are publicly available through the [Open Science Framework](#). For the present work, we converted the behavioral data from Experiment 1 to the Brain Imaging Data Structure (BIDS) format. For implementational details, please refer to *abm\_preparation.py*.

Experiment 1 comprised four dual-task combinations, each pairing a ‘memory-search’ task with a ‘classification’ task. The local BIDS dataset contains 49 participant records, whereas the article reports 48 recruited participants and an analyzed sample of 46 after exclusions (Br uning et al., 2020, p. 1415). Both component tasks involved either spatial or verbal stimuli, yielding two combinations with the same processing code and two with different processing codes. The ‘memory-search’ tasks required participants to recognize previously memorized patterns or letter pairs, whereas the ‘classification’ tasks required judgments about the rotation and reflection of three-dimensional objects or the parity of digit pairs. All tasks were answered manually, allowing the authors to vary competition for perceptual-cognitive resources while holding response modality constant. Experiment 2 extended this design by introducing vocal responses to the verbal ‘memory-search’ task.

### The PTD-RMC-FCDT paradigm

Here, we restrict our analysis to the combination of spatial ‘memory search’ and spatial ‘classification’ in Experiment 1. In this combination, participants decided whether a displayed pattern matched one of three previously memorized target patterns and whether two three-dimensional objects differed by rotation alone or by rotation and reflection. We refer to these component tasks as *pattern target detection* (PTD) and *rotation/mirror classification* (RMC), respectively, and to their combination as PTD-RMC. Participants organized their own task choices, with both task stimuli continuously visible. Each response immediately advanced the corresponding task stream, while the stimulus of the other task remained available for inspection. Responses were made with separate hands, with task-hand assignment counterbalanced across participants. Data collection comprised three runs, each containing two 120-second dual-task blocks, and stimuli were drawn from the respective stimulus sets without immediate repetition (cf. Figure 1, left).

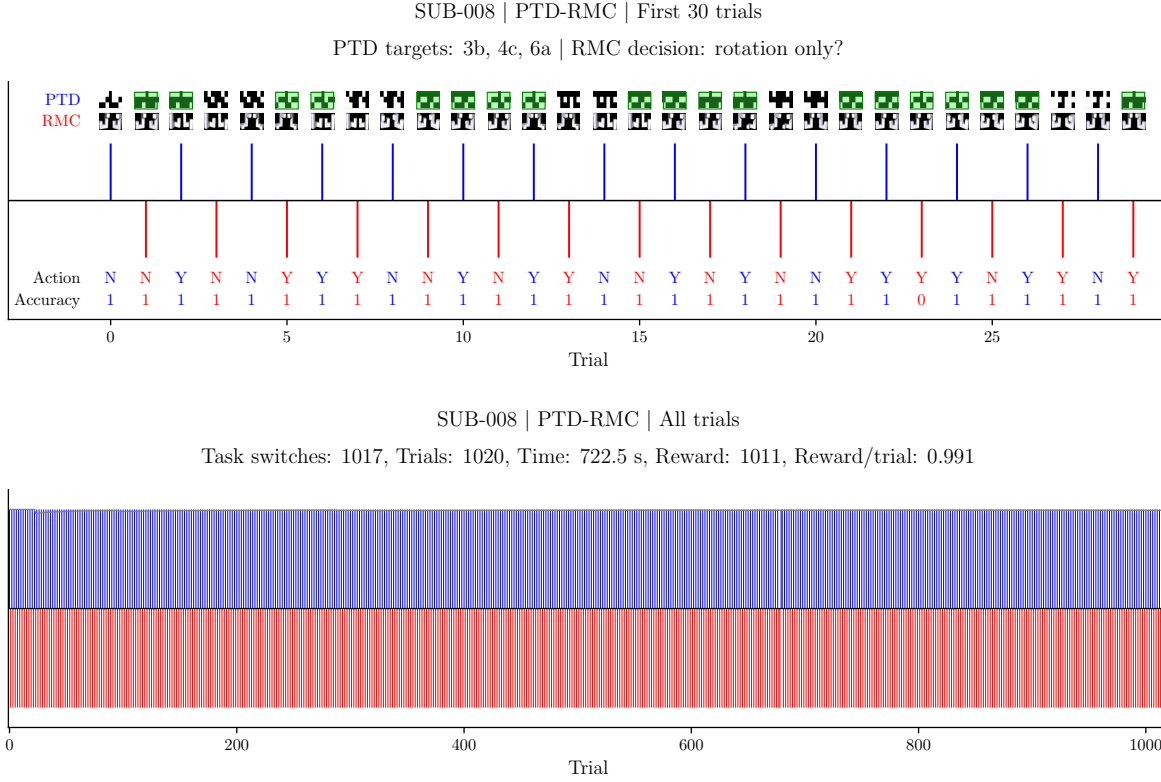


**Figure 1.** Experimental design of the PTD-RMC paradigm. Left: Exemplary trial stimuli and hand-specific yes/no response mappings. Right: Complete stimulus pools. Green marks one of the three PTD targets memorized before each block. For details, please refer to *abm\_figure\_1.py*.

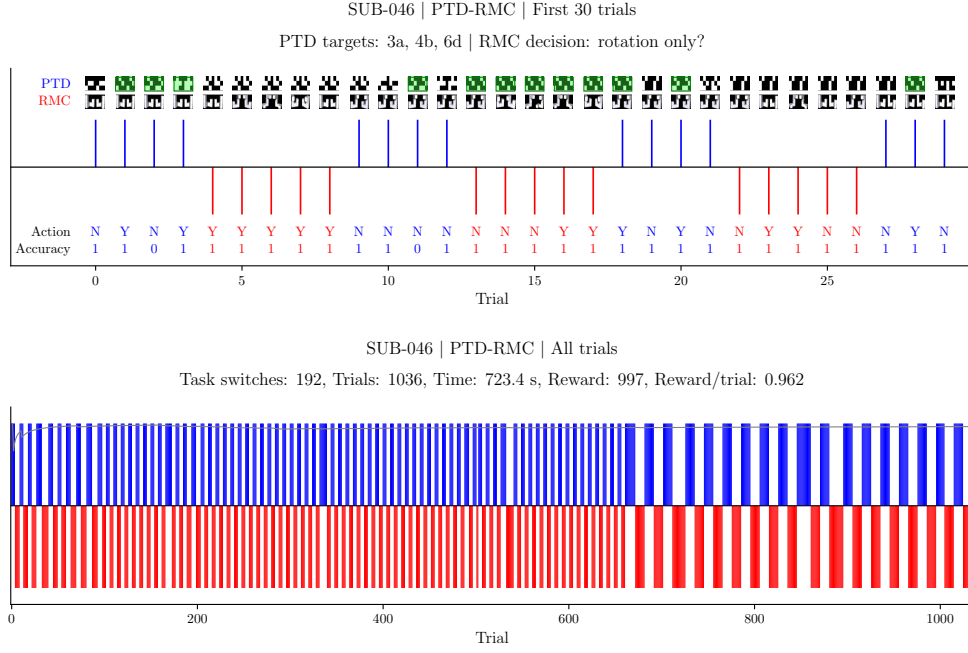
In more detail, the PTD task required participants to memorize three 2D patterns, presented as identical pairs for 15 s before each task block. Subsequent probes were drawn from a set of 18 patterns, and participants indicated whether the displayed pair matched one of the memorized targets. Targets and non-targets each occurred with a probability of 50%. In the RMC task, four baseline 3D objects yielded eight stimulus pairs, with the right object depicting either the left object rotated by 50 degrees or its mirror image rotated by 50 degrees. Both types occurred with a probability of 50%, so chance accuracy was 0.50 in both tasks (Brüning et al., 2020, pp. 1415–1416) (cf. Figure 1, right).

Regarding task allocation, participants were “instructed to not neglect or prefer one of the tasks compared to the other” (Brüning et al., 2020, p. 1416). Regarding performance, they were “instructed to maximize their amount of correct responses in the given time, that is, to perform the tasks according to an optimum speed-accuracy balance” (Brüning et al., 2020, pp. 1416–1417). No particular switching strategy was prescribed. At the end of every block, feedback was displayed for 5 s and reported, separately for each task, the number of performed trials and the number and percentage of correct responses (Brüning et al., 2020, p. 1417). Participants thus received information about both response throughput and accuracy for each task, rather than a single combined performance score. In addition to a fixed payment or course credit, they received a monetary bonus for each correctly answered stimulus, which could amount to EUR 10 in total (Brüning et al., 2020, p. 1415). The article does not specify the payment per correct response or a numerical penalty for task imbalance or switching.

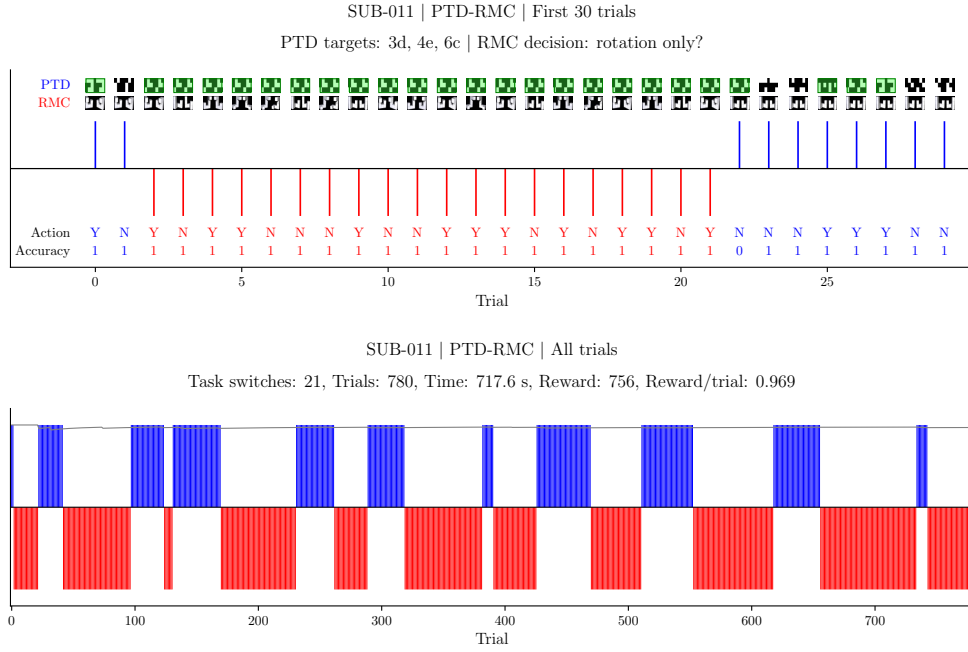
In the authors’ analyzed sample of 46 participants, the classification under high resource competition, which included this task combination, identified approximately 25 ‘blockers’, 15 ‘switchers’, and 6 ‘response groupers’, illustrating the response strategies described in the Introduction (Brüning et al., 2020, Figure 3). For implementational details, please refer to *abm\_description.py*.



**Figure 2A.** Behavior of participant 008 on the PTD-RMC task. The upper panel depicts the task dynamics during the first 30 trials of the first PTD-RMC block. The lower panel depicts all trials across all PTD-RMC blocks. Trial indices start at 0 in both panels, with the upper panel showing trials 0 to 29 and the lower panel counting continuously across blocks. In the upper panel, the first two rows of icons annotated PTD (blue) and RMC (red) depict the stimuli presented on each trial. The PTD row shows the pattern stimuli. The participant first memorized three target patterns, labeled 3b, 4c, and 6a in this block, whose occurrences are emphasized in green. The RMC row shows the object pairs to be classified as rotation only or as rotated and mirrored. The stem plot depicts the participant’s task choices. Upward-pointing blue stems indicate PTD responses, and downward-pointing red stems indicate RMC responses. The Action row shows the participant’s answer, with N denoting no and Y denoting yes. The Accuracy row shows 0 for an incorrect answer and 1 for a correct answer. The lower panel shows task choices across all trials and the cumulative proportion of correct answers as a gray line. Its annotations report task switches, trial count, total trial time, rewarded trials, and reward per trial. This participant alternated almost continuously between tasks, switching 1,017 times across 1,020 trials. For details, please refer to *abm\_figure\_2.py*.



**Figure 2B.** Behavior of participant 046 on the PTD-RMC task. The participant used short task sequences and switched tasks 192 times across 1,036 trials. For details, please refer to *abm\_figure\_2.py*.



**Figure 2C.** Behavior of participant 011 on the PTD-RMC task. The participant adopted a blocking strategy and switched tasks only 21 times across 780 trials. For details, please refer to *abm\_figure\_2.py*.

## Agentic behavioral model

Following Ostwald et al. (2026), we distinguish three components of an agentic behavioral model: a *task model*, an *agent model*, and a *data model*. The task model describes the experimental environment by specifying its possible states, the actions available to participants, and the consequences of those actions for subsequent states and performance outcomes. It thus provides a mathematical formulation of the rules governing the experiment. The agent model describes a computational approach to performing the task and serves as a hypothesis about the cognitive mechanisms underlying participants' behavior. It specifies the agent's knowledge of the task, its objectives, and the processes through which these inform its decisions. The data model connects this account to observed behavior by embedding the task and agent models within a statistical framework. This connection enables the estimation of model parameters and the comparison of competing accounts of task performance. Together, the three components make explicit the experimental conditions, the hypothesized cognitive processes, and the assumptions through which these hypotheses become amenable to empirical evaluation.

## Notation

In general, we use time-indexed lower-case letters for discrete-time random variables and non-indexed lower-case letters for the values they take. On occasion, a time-indexed lower-case symbol denotes a function. This use is clear from context and an explicit function definition. Thus, statements such as  $x_t = x$  mean that the random variable  $x_t$  takes on the value  $x$ . The corresponding upper-case letter denotes the set of values a random variable can take, with a time index added when this set varies over time. Collections of random variables are indicated by colon notation, i.e., for integer indices  $0 \leq t \leq t'$ , write

$$x_{t:t'} := (x_t, x_{t+1}, \dots, x_{t'}). \quad (1)$$

A collection with an end index smaller than its start index is empty. We use  $p$  for probability mass functions and overload it for joint, marginal, and conditional mass functions, as identified by its arguments. Expectations are used only when the relevant variable is integrable. Conditional probabilities and conditional expectations given a value are used only when that value has positive probability, unless a particular version of the conditional distribution or expectation has been specified (we understand a *version* as a concrete assignment of a conditional distribution or expectation to every conditioning value, including zero-probability values at which the joint distribution does not determine it. Different versions agree at positive-probability values but may differ at zero-probability values).

To keep probability calculations compact, we suppress explicit assignments of random variables to their values inside probability expressions and conditional expectations. Explicit assignments are retained when needed to specify particular component events, transformed values, or a value denoted by a distinct summation or maximization variable.

In statements such as “for all  $x_t \in X$ ”, the symbol  $x_t$  ranges over values in  $X$ . This is shorthand for “for all  $x \in X$ , with  $x_t = x$  substituted in the displayed expressions”. Such statements include values with zero probability under the distribution in question whenever the relevant functions or conditional distributions have been specified at those values.

When a time-indexed random-variable symbol appears below a summation sign, it is used as a dummy variable and takes each possible value of the corresponding random variable in turn. Thus we write, for example,

$$p(x_t) = \sum_{x' \in X} p(x_{t+1} = x', x_t) =: \sum_{x_{t+1}} p(x_{t+1}, x_t), \quad (2)$$

and

$$\mathbb{E}(y_{t+1}|x_t) = \sum_{y \in Y} y p(y_{t+1} = y|x_t) =: \sum_{y_{t+1}} y_{t+1} p(y_{t+1}|x_t). \quad (3)$$

For a function  $f$  defined on a set  $X$  and a random variable  $x_t$  taking values in  $X$ ,  $f(x_t)$  denotes the random variable obtained by applying  $f$  to  $x_t$ . On the event  $x_t = x$ , this random variable takes the value  $f(x)$ . The same convention applies to functions of several random variables.

## Task model

We define the participant- and block-specific task model as

$$\mathcal{T} := (J, S, A, R_{\mathcal{T}}, p(s_0), p(s_{t+1}|s_t, a_t), \varrho). \quad (4)$$

The task-model tuple uses concepts from the theory of Markov decision processes discussed in S1 (Definition 2).

## Time indices

The trial- and action index set is

$$J := \{0, 1, \dots, n\}, \quad (5)$$

where  $n$  is the observed action count within the fixed time window. It may vary across participants and blocks but is held constant in each task model. Actions are selected at stages  $t = 0, \dots, n-1$ , and stage  $n$  is the horizon boundary. Note that  $n$  denotes the horizon represented by  $T$  in S1. Here, the symbol  $T$  is reserved for the task set  $\{1, 2\}$ , introduced below. The model conditions on the observed number of responses within a block of fixed duration. It describes choices over these  $n$  decision stages without modeling the response times that determine how many responses fit into the available time. The horizon boundary ends the decision sequence without requiring a special terminal state. The boundary self-loops introduced below complete the conditional state distribution and do not determine when the block ends.

## State space $\mathcal{S}$

The state records the two task stimuli, the previously executed task, and the current task imbalance. We first define these components and then assemble the complete state space.

### Labeled stimulus sets

For  $j = 1, 2$ , let  $I_j := \mathbb{N}_{n_j}$ , where  $\mathbb{N}_m := \{1, \dots, m\}$ , denote the unique identifiers of the visual stimuli of Task  $j$  as depicted in Table 1.

For tasks  $j = 1, 2$ , let

$$\kappa_j : I_j \rightarrow \{0, 1\}, i \mapsto \kappa_j(i) \quad (6)$$

denote the participant- and block-specific label function. For Task 1,  $\kappa_1$  differs between participants and blocks, as it indicates whether a visual pattern belonged to the memorized target set or not. For Task 2,  $\kappa_2$  is identical across participants and blocks, as it indicates whether the right object is rotated only or both mirrored and rotated relative to the left object. Note that to keep the notational complexity at bay, we leave the participant and block dependence of  $\kappa_j$ , and hence each task model, implicit. The participant- and block-specific labeled stimulus sets are then defined as

$$S^j := \{(\kappa_j(i), i) \mid i \in I_j\}, \quad (7)$$

such that for  $j = 1, 2$

$$s^j := (s^{j,1}, s^{j,2}) \in S^j. \quad (8)$$

Here,  $s^{j,1} \in \{0, 1\}$  denotes the task-relevant binary stimulus label,  $s^{j,2} \in I_j$  denotes the unique visual stimulus identifier, and  $s^{j,1} = \kappa_j(s^{j,2})$  by definition. Note that including the label alongside the identifier adds no independent state information. It makes the task-relevant category explicit for the reward function and the subsequent agent-model reduction, while the identifier retains the stimulus identity needed to exclude direct repetitions. Only label-identifier pairs consistent with  $\kappa_j$  belong to  $S^j$ . The label  $s^{j,1} = 1$  denotes the task-defined positive stimulus category, whereas  $s^{j,1} = 0$  denotes the complementary stimulus category. Specifically, label 1 denotes a target in PTD and rotation only in RMC. Label 0 denotes a non-target in PTD and mirrored-and-rotated objects in RMC.

Note that because the labeling of the visual stimuli merely induces a partition of the stimulus identifier sets, the cardinality of the labeled stimulus space  $S^j$  corresponds to the cardinality of the stimulus identifier set, i.e.,

$$|S^j| = |I_j| = |\mathbb{N}_{n_j}| = n_j \text{ for } j = 1, 2. \quad (9)$$

For later reference, we define the label-specific subsets of  $I_j$  and their cardinalities as

$$I_{j,l} := \{i \in I_j \mid \kappa_j(i) = l\} \subset I_j \text{ for } l \in \{0, 1\} \quad (10)$$

| Task | $n_j$ | $I_j$             | Label $l$ | Category            | $n_{j,l}$ |
|------|-------|-------------------|-----------|---------------------|-----------|
| PTD  | 18    | $\mathbb{N}_{18}$ | 1         | Target              | 3         |
|      |       |                   | 0         | Non-target          | 15        |
| RMC  | 8     | $\mathbb{N}_8$    | 1         | Rotation only       | 4         |
|      |       |                   | 0         | Rotation and mirror | 4         |

**Table 1.** PTD-RMC stimulus sets and label-specific cardinalities. PTD identifiers denote patterns, and RMC identifiers denote object pairs.

and

$$n_{j,l} := |I_{j,l}| \text{ for } l \in \{0, 1\} \text{ such that } n_{j,0} + n_{j,1} = n_j, \quad (11)$$

respectively. Table 1 summarizes the stimulus counts, identifier sets, category meanings, and label-specific cardinalities.

#### *Previous-task identity*

The task set is defined as

$$T := \{1, 2\} \quad (12)$$

Here 1 denotes Task 1 execution and 2 denotes Task 2 execution. The state component  $s^3$  records which task was executed most recently and takes values in  $T$  after the first action.

#### *Task imbalance*

The task imbalance set is defined as

$$B := \{-n, -n + 1, \dots, n - 1, n\}, \quad (13)$$

with  $n$  as defined in the [time indices section](#). The task imbalance cannot exceed  $n$  in absolute value, and its cardinality is  $|B| = 2n + 1$ . Positive values of  $s^4$  indicate a surplus of Task 1 actions, negative values indicate a surplus of Task 2 actions, and zero indicates equal numbers of actions on both tasks.

#### *Complete state space*

Combining the components above, we distinguish the initial state layer

$$S_0 := S^1 \times S^2 \times \{\emptyset\} \times \{0\}, \quad (14)$$

from the regular state space

$$S_+ := S^1 \times S^2 \times T \times B. \quad (15)$$

The complete state space is

$$S := S_0 \cup S_+ \quad (16)$$

| Action | Task judgment           | Answer  |
|--------|-------------------------|---------|
| (1, 1) | Task 1 (Target?)        | 1 (Yes) |
| (1, 0) | Task 1 (Target?)        | 0 (No)  |
| (2, 1) | Task 2 (Rotation only?) | 1 (Yes) |
| (2, 0) | Task 2 (Rotation only?) | 0 (No)  |

**Table 2.** Action set value interpretation

with states of the form

$$s := ((s^{1,1}, s^{1,2}), (s^{2,1}, s^{2,2}), s^3, s^4). \quad (17)$$

Before the first action,  $s^3 = \emptyset$  indicates that no task has yet been executed and  $s^4 = 0$  records zero task imbalance. This combination occurs only in  $S_0$ . The first action establishes the previous-task identity and therefore does not constitute a task switch. Subsequent states belong to  $S_+$ .

#### *State-space cardinality*

Because  $S_0$  and  $S_+$  are disjoint, the state space cardinality is

$$|S| = |S_0| + |S_+| = |S^1| \cdot |S^2| (1 + |T| \cdot |B|). \quad (18)$$

Across the 294 PTD-RMC blocks in the current BIDS dataset, the maximum observed action count is 245. Thus, the largest imbalance set is of cardinality  $|B| = 2 \cdot 245 + 1 = 491$  which for the state space yields a cardinality of

$$|S| = 18 \cdot 8 \cdot (1 + 2 \cdot 491) = 141,552. \quad (19)$$

#### **Action set**

On each trial, the participant can respond in either task with either a Yes or a No answer. We represent each action as an ordered pair and define the action set

$$A := \{1, 2\} \times \{0, 1\}, \quad (20)$$

where the first component identifies the selected task and the second component encodes the binary answer, with 0 denoting No and 1 denoting Yes. The resulting value interpretations are listed in Table 2.

For later reference, we define the task-specific action sets by

$$A^j := \{j\} \times \{0, 1\} \text{ for } j = 1, 2, \quad (21)$$

such that

$$A = A^1 \cup A^2 \text{ and } A^1 \cap A^2 = \emptyset. \quad (22)$$

For the change in task imbalance, we define the *task imbalance increment function*

$$\Delta : A \rightarrow \{-1, 1\}, a \mapsto \Delta(a) := \begin{cases} +1 & (a \in A^1) \\ -1 & (a \in A^2) \end{cases} \quad (23)$$

### Initial-state distribution

At block onset, the stimulus label and identifier pairs of the two task streams are assumed to be mutually independent. Each stimulus label is uniformly distributed over  $\{0, 1\}$ , and, conditional on its label, each identifier is uniformly distributed over the corresponding set  $I_{j,l}$ . A discrete uniform distribution on a finite set assigns equal probability to every element of that set and zero probability to all other values. Here, the two labels therefore each have probability  $1/2$ , and each identifier in  $I_{j,l}$  has conditional probability  $1/n_{j,l}$ .

We use the Iverson bracket  $\llbracket P \rrbracket$  to denote one when the condition  $P$  holds and zero otherwise. Note that there is no previous stimulus at block onset, so direct-repetition exclusion does not apply to the initial identifiers. The initial-state convention is encoded by restricting the distribution to  $S_0$ . We thus define the initial state distribution as

$$p(s_0) := \prod_{j=1}^2 \left( \frac{1}{2} \frac{\llbracket s_0^{j,2} \in I_{j,s_0^{j,1}} \rrbracket}{n_{j,s_0^{j,1}}} \right) \llbracket s_0^3 = \emptyset \rrbracket \llbracket s_0^4 = 0 \rrbracket, s_0 \in S. \quad (24)$$

Consequently,  $p(s_0)$  has support only on  $S_0$ .

### State-action conditional state distribution

The state-action conditional state distribution describes how a response changes the experimental state. Following S1, we use  $s_t$ ,  $a_t$ , and  $s_{t+1}$  for the current state, action, and successor-state values, suppressing explicit assignments inside probabilities. The distribution is Markovian and time homogeneous: the next state depends on these values without additional dependence on the preceding history or decision stage.

#### *Construction of the next state*

Consider an action  $a_t \in A^j$  selecting Task  $j$  in a state with  $|s_t^4| < n$ . The selected task receives a new stimulus. First, its binary label is sampled with probability  $1/2$  for either value, independently of the current stimulus. Note that the labels are equally likely even when their identifier sets differ in size,

as for PTD targets and non-targets. Conditional on the new label  $l$ , the new identifier is sampled uniformly from  $I_{j,l} \setminus \{s_t^{j,2}\}$ . Thus, direct repetition of the current stimulus is excluded. If the current identifier belongs to  $I_{j,l}$ , the admissible set contains  $n_{j,l} - 1$  identifiers. Otherwise, all  $n_{j,l}$  identifiers remain admissible. These two possibilities give the denominator  $n_{j,l} - \mathbb{I}[s_t^{j,2} \in I_{j,l}]$  in the conditional probabilities below.

The remaining state components update deterministically. The stimulus of the unselected task remains unchanged, the previous-task component becomes  $j$ , and task imbalance changes by  $\Delta(a_t)$ , increasing by one for Task 1 and decreasing by one for Task 2. These updates depend on the selected task and apply whether the response to its stimulus is correct or incorrect.

#### *Boundary convention*

The conditional distribution is prescribed for every state-action pair in  $S \times A$ , including combinations that cannot occur before the horizon boundary. At  $|s_t^4| = n$ , we let either action leave the state unchanged. These self-loops keep the next state within  $S$ , including when an imbalance increment would otherwise take it outside  $B$ . Starting from  $S_0$ , absolute imbalance can increase by at most one per action, so this boundary cannot be reached before the final action. No further action is selected then. Consequently, the self-loops complete the mathematical specification without affecting trajectories over the  $n$  decision stages. The horizon determines when the block ends, rather than the boundary convention.

Combining the stimulus sampling, deterministic updates, and boundary convention gives the following conditional probabilities:

$$p(s_{t+1}|s_t, a_t) := \begin{cases} 1 & |s_t^4| = n \wedge s_{t+1} = s_t \\ \frac{1}{2} \frac{\mathbb{I}[s_{t+1}^{j,2} \neq s_t^{j,2}] \mathbb{I}[s_{t+1}^{j,2} \in I_{j,s_{t+1}^{j,1}}]}{n_{j,s_{t+1}^{j,1}} - \mathbb{I}[s_t^{j,2} \in I_{j,s_{t+1}^{j,1}}]} & |s_t^4| < n \wedge a_t \in A^j \wedge s_{t+1}^3 = j \wedge s_{t+1}^{3-j} = s_t^{3-j} \wedge s_{t+1}^4 = s_t^4 + \Delta(a_t) \text{ for } j = 1, 2 \\ 0 & \text{else} \end{cases} \quad (25)$$

#### *Consistency with the task construction*

Theorem 1 verifies that the conditional probabilities in (25) implement the construction above. Property (1) establishes normalization and equal label probabilities. Property (2) establishes uniform identifier sampling with direct repetition excluded. Property (3) establishes the unchanged unselected stimulus and the deterministic updates of previous-task identity and task imbalance.

**Theorem 1** (Consistency of the state-action conditional state distribution). *For  $j \in \{1, 2\}$ , assume  $n_{j,l} > 1$  for both  $l \in \{0, 1\}$ . For any state  $s_t$  with  $|s_t^4| < n$  and any action  $a_t \in A^j$ , the specified conditional probabilities form a probability mass function that encodes the following properties:*

1. The conditional probabilities sum to one. Conditional on the selected action, the label of the new stimulus in the selected task stream is independent of the previous stimulus and takes either value with probability  $1/2$ , i.e.,

$$p(s_{t+1}^{j,1} = l | s_t, a_t) = \frac{1}{2} \text{ for } l \in \{0, 1\}. \quad (26)$$

2. Conditional on the new label, the previous state, and the selected action, the new identifier is distributed uniformly over the corresponding label-specific identifier set with the previous identifier excluded. This admissible set is  $I_{j,l} \setminus \{s_t^{j,2}\}$  and contains  $n_{j,l} - \mathbb{I}[s_t^{j,2} \in I_{j,l}]$  identifiers, which need not be consecutive integers. Its conditional probability mass function depends on the previous state only through the previous identifier  $s_t^{j,2}$ , i.e., for  $l \in \{0, 1\}$  and  $i \in I_j$ ,

$$p(s_{t+1}^{j,2} = i | s_{t+1}^{j,1} = l, s_t, a_t) = \begin{cases} \frac{1}{n_{j,l} - \mathbb{I}[s_t^{j,2} \in I_{j,l}]} & i \in I_{j,l} \setminus \{s_t^{j,2}\}, \\ 0 & \text{otherwise.} \end{cases} \quad (27)$$

3. For any state  $s_t$  with  $|s_t^4| < n$  and action  $a_t \in A^j$ , the stimulus of the unselected task remains unchanged, the previously executed task becomes  $j$ , and the task imbalance changes by  $\Delta(a_t)$ . Specifically,

$$p(s_{t+1}^{3-j} = s_t^{3-j}, s_{t+1}^3 = j, s_{t+1}^4 = s_t^4 + \Delta(a_t) | s_t, a_t) = 1. \quad (28)$$

■

*Proof.* We show each property in turn.

*Proof of Property (1)*

For a fixed  $j \in \{1, 2\}$ ,  $l \in \{0, 1\}$ , state  $s_t$  with  $|s_t^4| < n$ , and action  $a_t \in A^j$ , we have

$$\begin{aligned} & p(s_{t+1}^{j,1} = l | s_t, a_t) \\ & \stackrel{(1)}{=} \sum_{s_{t+1}^{j,2} \in I_j} \sum_{s_{t+1}^{3-j} \in S^{3-j}} \sum_{s_{t+1}^3 \in T} \sum_{s_{t+1}^4 \in B} p(s_{t+1}^{j,1} = l, s_{t+1}^{j,2}, s_{t+1}^{3-j}, s_{t+1}^3, s_{t+1}^4 | s_t, a_t) \\ & \stackrel{(2)}{=} \sum_{s_{t+1}^{j,2} \in I_j} p(s_{t+1}^{j,1} = l, s_{t+1}^{j,2}, s_{t+1}^{3-j} = s_t^{3-j}, s_{t+1}^3 = j, s_{t+1}^4 = s_t^4 + \Delta(a_t) | s_t, a_t) \\ & \stackrel{(3)}{=} \sum_{s_{t+1}^{j,2} \in I_j} \frac{1}{2} \frac{\mathbb{I}[s_{t+1}^{j,2} \neq s_t^{j,2}] \mathbb{I}[s_{t+1}^{j,2} \in I_{j,l}]}{n_{j,l} - \mathbb{I}[s_t^{j,2} \in I_{j,l}]} \\ & \stackrel{(4)}{=} \frac{1}{2} \frac{1}{n_{j,l} - \mathbb{I}[s_t^{j,2} \in I_{j,l}]} \sum_{s_{t+1}^{j,2} \in I_j} \mathbb{I}[s_{t+1}^{j,2} \neq s_t^{j,2}] \mathbb{I}[s_{t+1}^{j,2} \in I_{j,l}] \\ & \stackrel{(5)}{=} \frac{1}{2} \frac{1}{n_{j,l} - \mathbb{I}[s_t^{j,2} \in I_{j,l}]} \begin{cases} \underbrace{1 + \dots + 1}_{n_{j,l}-1 \text{ terms}}, & s_t^{j,2} \in I_{j,l}, \\ \underbrace{1 + \dots + 1}_{n_{j,l} \text{ terms}}, & s_t^{j,2} \notin I_{j,l} \end{cases} \\ & \stackrel{(6)}{=} \frac{1}{2} \frac{1}{n_{j,l} - \mathbb{I}[s_t^{j,2} \in I_{j,l}]} (n_{j,l} - \mathbb{I}[s_t^{j,2} \in I_{j,l}]) \end{aligned}$$

$$\stackrel{(7)}{=} \frac{1}{2}.$$

Here, (1) marginalizes over every component of the next state other than the fixed label  $s_{t+1}^{j,1} = l$ . Although the sum ranges over all identifiers in  $I_j$ , an identifier incompatible with  $l$  forms no element of the labeled stimulus set  $S^j$  and therefore contributes zero. Equivalently, this is enforced by the label-set Iverson bracket in (3). In (2), the conditional state distribution assigns probability zero to every term except those with  $s_{t+1}^{3-j} = s_t^{3-j}$ ,  $s_{t+1}^3 = j$ , and  $s_{t+1}^4 = s_t^4 + \Delta(a_t)$ , so the three corresponding sums collapse. Step (3) substitutes the second branch of the conditional state distribution and uses the fixed new label  $s_{t+1}^{j,1} = l$ . Step (4) moves all factors that do not depend on the remaining summation variable  $s_{t+1}^{j,2}$  outside the sum. Step (5) evaluates that sum by cases: if the previous identifier  $s_t^{j,2}$  belongs to  $I_{j,l}$ , one of its  $n_{j,l}$  identifiers is excluded and  $n_{j,l} - 1$  unit terms remain. Otherwise, all  $n_{j,l}$  unit terms remain. Step (6) expresses these two cases compactly as  $n_{j,l} - \llbracket s_t^{j,2} \in I_{j,l} \rrbracket$ . Finally, (7) cancels this positive quantity with the identical denominator, where positivity follows from the assumption  $n_{j,l} > 1$ .

Thus the new label has probability  $1/2$  for every previous state  $s_t$  and, conditional on the selected action  $a_t \in A^j$ , is independent of the previous stimulus. Moreover, all transition probabilities are nonnegative because their numerators are products of Iverson brackets, while their denominators satisfy  $n_{j,l} - \llbracket s_t^{j,2} \in I_{j,l} \rrbracket \geq n_{j,l} - 1 > 0$ . Finally, summing the derived marginal probability over the two labels gives

$$\sum_{s_{t+1} \in S} p(s_{t+1} | s_t, a_t) = \sum_{l \in \{0,1\}} p(s_{t+1}^{j,1} = l | s_t, a_t) = \frac{1}{2} + \frac{1}{2} = 1. \quad (29)$$

Hence, the specified conditional probabilities form a probability mass function.

*Proof of Property (2)*

For fixed  $j \in \{1, 2\}$ ,  $l \in \{0, 1\}$ ,  $i \in I_j$ , a state  $s_t$  with  $|s_t^4| < n$ , and an action  $a_t \in A^j$ , we have

$$\begin{aligned} & p(s_{t+1}^{j,2} = i | s_{t+1}^{j,1} = l, s_t, a_t) \\ & \stackrel{(1)}{=} \frac{p(s_{t+1}^{j,2} = i, s_{t+1}^{j,1} = l | s_t, a_t)}{p(s_{t+1}^{j,1} = l | s_t, a_t)} \\ & \stackrel{(2)}{=} \frac{\sum_{s_{t+1}^{3-j} \in S^{3-j}} \sum_{s_{t+1}^3 \in T} \sum_{s_{t+1}^4 \in B} p(s_{t+1}^{j,1} = l, s_{t+1}^{j,2} = i, s_{t+1}^{3-j}, s_{t+1}^3, s_{t+1}^4 | s_t, a_t)}{\sum_{s_{t+1}^{j,2} \in I_j} \sum_{s_{t+1}^{3-j} \in S^{3-j}} \sum_{s_{t+1}^3 \in T} \sum_{s_{t+1}^4 \in B} p(s_{t+1}^{j,1} = l, s_{t+1}^{j,2}, s_{t+1}^{3-j}, s_{t+1}^3, s_{t+1}^4 | s_t, a_t)} \\ & \stackrel{(3)}{=} \frac{p(s_{t+1}^{j,1} = l, s_{t+1}^{j,2} = i, s_{t+1}^{3-j} = s_t^{3-j}, s_{t+1}^3 = j, s_{t+1}^4 = s_t^4 + \Delta(a_t) | s_t, a_t)}{\sum_{s_{t+1}^{j,2} \in I_j} p(s_{t+1}^{j,1} = l, s_{t+1}^{j,2}, s_{t+1}^{3-j} = s_t^{3-j}, s_{t+1}^3 = j, s_{t+1}^4 = s_t^4 + \Delta(a_t) | s_t, a_t)} \\ & \stackrel{(4)}{=} \frac{\frac{1}{2} \llbracket i \neq s_t^{j,2} \rrbracket \llbracket i \in I_{j,l} \rrbracket}{\frac{1}{2} n_{j,l} - \llbracket s_t^{j,2} \in I_{j,l} \rrbracket} \\ & \stackrel{(5)}{=} \begin{cases} \frac{1}{\sum_{s_{t+1}^{j,2} \in I_j} \llbracket s_{t+1}^{j,2} \neq s_t^{j,2} \rrbracket \llbracket s_{t+1}^{j,2} \in I_{j,l} \rrbracket}}, & i \in I_{j,l} \setminus \{s_t^{j,2}\}, \\ 0, & i \notin I_{j,l} \setminus \{s_t^{j,2}\}. \end{cases} \\ & \stackrel{(6)}{=} \begin{cases} \frac{1}{\sum_{s_{t+1}^{j,2} \in I_{j,l}} \llbracket s_{t+1}^{j,2} \neq s_t^{j,2} \rrbracket}}, & i \in I_{j,l} \setminus \{s_t^{j,2}\}, \\ 0, & i \notin I_{j,l} \setminus \{s_t^{j,2}\}. \end{cases} \end{aligned}$$

$$\stackrel{(7)}{=} \begin{cases} \frac{1}{n_{j,l} - \mathbb{I}[s_t^{j,2} \in I_{j,l}]}, & i \in I_{j,l} \setminus \{s_t^{j,2}\}, \\ 0, & i \notin I_{j,l} \setminus \{s_t^{j,2}\} \end{cases}$$

Here, (1) is the definition of conditional probability. Its denominator is  $1/2$  by Property (1) and is therefore strictly positive. In (2), the numerator is marginalized over the unselected task stimulus, the previously executed task, and the task imbalance, because the new label  $l$  and new identifier  $i$  are fixed. The denominator is additionally marginalized over the new identifier  $s_{t+1}^{j,2} \in I_j$ , because only the new label  $l$  remains fixed there. In (3), the conditional state distribution makes every term zero except those satisfying  $s_{t+1}^{3-j} = s_t^{3-j}$ ,  $s_{t+1}^3 = j$ , and  $s_{t+1}^4 = s_t^4 + \Delta(a_t)$ , so the corresponding sums in both numerator and denominator collapse. Step (4) substitutes the second branch of the conditional state distribution, with the fixed new label  $s_{t+1}^{j,1} = l$ , into the remaining conditional probabilities and moves the factors that do not depend on  $s_{t+1}^{j,2}$  in front of the denominator sum. Step (5) distinguishes whether  $i$  belongs to the admissible support  $I_{j,l} \setminus \{s_t^{j,2}\}$ . On this support, both numerator Iverson brackets equal one. Outside it, at least one equals zero. The common factors  $1/2$  and  $1/(n_{j,l} - \mathbb{I}[s_t^{j,2} \in I_{j,l}])$  are cancelled between the numerator and denominator. In (6), the label-membership Iverson bracket restricts the effective summation domain from  $I_j$  to  $I_{j,l}$  and is then omitted. Step (7) evaluates the remaining sum as  $n_{j,l} - \mathbb{I}[s_t^{j,2} \in I_{j,l}]$ . Its reciprocal is the uniform probability assigned to each admissible identifier in Property (2). The result depends on the previous stimulus only through its identifier  $s_t^{j,2}$ . Note that the previous label carries no information beyond the identifier, because  $s_t^{j,1} = \kappa_j(s_t^{j,2})$  by the definition of  $S^j$ . This proves Property (2).

*Proof of Property (3)*

For  $|s_t^4| < n$  and  $a_t \in A^j$ , the conditional state distribution assigns positive probability only to successor states satisfying

$$s_{t+1}^{3-j} = s_t^{3-j}, s_{t+1}^3 = j, s_{t+1}^4 = s_t^4 + \Delta(a_t). \quad (30)$$

All other successor states have probability zero. Since the transition probabilities sum to one by Property (1), the displayed updates hold jointly with probability one.

□

As a descriptive validation of the experimental state model, we compared the observed PTD and RMC transition frequencies with frequencies generated from the full MDP under the original participant-specific experimental designs. The simulated frequencies closely reproduce the principal empirical label and identifier patterns. The validation and its four figures are reported in [Supplement S2](#).

## Task reward and performance feedback

The task reward represents the performance quantities reported to participants at the end of each block. This feedback displayed the number of completed trials and the number and percentage of correct responses separately for each task for 5 s (Brüning et al., 2020, p. 1417). To recover these block-level quantities from individual actions, we record task completion and response correctness in a four-component outcome vector. This trial-based representation does not imply that feedback was displayed after every response. The corresponding reward set is

$$R_{\mathcal{T}} := \{(1, 1, 0, 0), (1, 0, 0, 0), (0, 0, 1, 1), (0, 0, 1, 0)\}. \quad (31)$$

The four components represent a completed Task 1 trial, a correct Task 1 response, a completed Task 2 trial, and a correct Task 2 response. Summing the first and third components over a block gives the number of completed trials for each task, while summing the second and fourth gives the corresponding number of correct responses. The percentage of correct responses is obtained by dividing each correct-response count by the corresponding trial count and multiplying by 100, provided that the task has been performed. The trial-based task reward function is

$$\varrho : S \times A \rightarrow R_{\mathcal{T}}, (s, (j, k)) \mapsto \varrho(s, (j, k)) := (\llbracket j = 1 \rrbracket, \llbracket j = 1 \rrbracket \llbracket k = s^{1,1} \rrbracket, \llbracket j = 2 \rrbracket, \llbracket j = 2 \rrbracket \llbracket k = s^{2,1} \rrbracket). \quad (32)$$

Note that the task reward is fully determined once the current state and selected action are specified. The state identifies the correct response for each task, and the action identifies both the selected task and the response given. Together, these determine which task was completed and whether the response was correct. The conditional reward distribution therefore assigns probability one to the corresponding outcome vector  $\varrho(s, a)$  and probability zero to every other reward value. Explicitly, it is

$$p(r_{t+1} | s_t, a_t) := \llbracket r_{t+1} = \varrho(s_t, a_t) \rrbracket. \quad (33)$$

This also determines the joint conditional distribution of the next state and reward, as introduced in S1. For a given current state and action, the probability of a particular next state together with the prescribed reward is simply the probability of that next state. Any combination involving a different reward has probability zero. Consequently,

$$p(r_{t+1}, s_{t+1} | s_t, a_t) = p(r_{t+1} | s_t, a_t) p(s_{t+1} | s_t, a_t). \quad (34)$$

Thus, specifying  $\varrho$  and the state-action conditional state distribution already specifies the joint state-reward distribution, which need not appear as a separate component of the task-model tuple. This conditional determinism does not imply that rewards are constant across trials. The outcome vector changes with the selected task and response accuracy, and rewards can be uncertain when the current state or selected action is not given.

The instructions and monetary incentives described for the [PTD-RMC-FCDT paradigm](#) motivate the performance quantities recorded by this reward. They do not specify numerical costs for task imbalance or switching. Accordingly,  $\varrho$  records task performance without assigning subjective values to these outcomes or costs to particular response strategies. Note that this vector-valued task reward differs from the scalar reward assumed in S1. The agent’s scalar subjective reward, which specifies its optimization goals, is introduced in the [subjective reward function section](#).

## Agent model A1

We define agent model A1 as

$$\mathcal{A}_1 := (\mathcal{T}, \mathcal{M}_\chi, D, p_{\beta, \sigma}(d_t | s_t)). \quad (35)$$

Here,  $\mathcal{T}$  is the task model,  $\mathcal{M}_\chi$  is the reduced FCDT-MDP used to derive the agent’s optimal task choices,  $D := A$  is the agent’s decision space, and  $p_{\beta, \sigma}(d_t | s_t)$  is its conditional decision distribution. The task model provides the state and action spaces needed to express reduced task choices as agent decisions. In particular, action lifting uses the stimulus labels in the task state to translate a selected task into the decision corresponding to its correct response. The reduced MDP combines the task dynamics relevant to planning with the agent’s subjective reward, whose parameters  $\beta$  and  $\sigma$  determine its imbalance and switching costs.

We first describe this subjective reward and the reasoning underlying the MDP reduction. We then specify  $\mathcal{M}_\chi$  and explain how its optimal policy determines the agent’s decisions.

### Subjective reward function

The subjective reward expresses the agent’s preference for correct responses and balanced task completion, together with the cost it assigns to switching between tasks. We represent these optimization goals by an additive reward function with imbalance parameter  $\beta \in \mathbb{R}$  and switching-cost parameter  $\sigma \geq 0$ ,

$$\rho_{\beta, \sigma} : S \times A \rightarrow \mathbb{R}, (s, a) \mapsto \rho_{\beta, \sigma}(s, a) := \rho(s, a) + \rho_\beta(s, a) + \rho_\sigma(s, a), \quad (36)$$

The three components specify how accuracy, task balance, and task switching contribute to the reward associated with a state and action.

#### *Task accuracy*

The accuracy component rewards correct responses in either task. It is defined by  $(s, a) \mapsto \rho(s, a)$ , with

$$\rho(s, a) := \mathbb{I}[(s^{1,1} = 0 \wedge a = (1, 0)) \vee (s^{1,1} = 1 \wedge a = (1, 1)) \vee (s^{2,1} = 0 \wedge a = (2, 0)) \vee (s^{2,1} = 1 \wedge a = (2, 1))] \quad (37)$$

This component contributes one when the selected response matches the stimulus label of the selected task and zero otherwise. It is the sum of the two correct-response components of the task reward  $\varrho$ .

#### *Task balance*

The balance component expresses a preference for completing similar numbers of trials in the two tasks. It is defined by  $(s, a) \mapsto \rho_\beta(s, a)$ , with

$$\rho_\beta(s, a) := -\beta \cdot |s^4 + \Delta(a)|, \quad (38)$$

For states reached before the horizon boundary,  $s^4 + \Delta(a)$  is the imbalance after the selected action. The component therefore assigns a value proportional to its absolute value, with  $\beta$  determining the strength of this preference. Positive values of  $\beta$  penalize task imbalance, whereas negative values reward it. We leave the sign of  $\beta$  unrestricted so that its direction is determined by the data rather than imposed a priori. The instruction to give both tasks equal priority motivates the balance component. Its linear form and weight are modeling assumptions.

#### *Task switching*

The switching component represents a cost of selecting a task different from the previously selected task. It is defined by  $(s, a) \mapsto \rho_\sigma(s, a)$ , with

$$\rho_\sigma(s, a) := -\sigma \cdot \llbracket (s^3 = 1 \wedge a \in A^2) \vee (s^3 = 2 \wedge a \in A^1) \rrbracket \quad (39)$$

A task switch contributes  $-\sigma$ , while repeating a task contributes zero. The initial action incurs no switching cost because  $s^3 = \emptyset$  in the initial state. Both the inclusion of a switching cost and its weight are assumptions about the agent's subjective reward.

Note that the accuracy component has a fixed unit weight and thus sets the scale for both  $\beta$  and  $\sigma$ . The reward is expressed in units of one correct response, without identifying that unit with a specified monetary payment. The imbalance and switching costs are subjective valuations, not penalties imposed by the experimenter. Their relation to task instructions and performance feedback is discussed in the [task reward section](#).

#### *Reward set and conditional reward*

For fixed parameters  $\beta$  and  $\sigma$ , the scalar subjective reward set is defined as

$$R := \{\rho_{\beta, \sigma}(s, a) : s \in S, a \in A\}. \quad (40)$$

The set  $R$  is finite because both  $S$  and  $A$  are finite. The reward is fully determined by the current state and selected action,

$$r_{t+1} = \rho_{\beta, \sigma}(s_t, a_t). \quad (41)$$

Consequently, the reward function coincides with the state-action conditional expected reward in the general MDP theory,

$$\rho_{\beta,\sigma}(s_t, a_t) = \mathbb{E}(r_{t+1} | s_t, a_t). \quad (42)$$

## MDP reduction

The agent combines the task dynamics with its subjective reward to form an unreduced internal MDP, denoted by  $\mathcal{M}_{\beta,\sigma}$ . This decision problem uses the scalar subjective reward rather than the task’s vector-valued performance outcomes. We reduce it in three steps, retaining the information needed to derive optimal decisions. The argument below explains each step intuitively. Theorem 2 defines both MDPs and states the preservation of optimal values and optimality under action lifting.

For the intermediate reductions, we consider the regular state space  $S_+$  introduced in the [task state-space section](#). The complete state space remains  $S = S_0 \cup S_+$ . The reduced initial state and the lifting of initial decisions are included below.

### *Removing stimulus identifiers*

The first step removes the stimulus identifiers while retaining both stimulus labels, the previously selected task, and the imbalance. None of the subjective reward components depends on the identifiers. They do, however, affect the state-action conditional state distribution, because the exclusion of direct repetitions removes the current identifier from the possible successors in the selected task stream.

The crucial distinction is that this exclusion changes which identifier can appear next, without changing the probabilities of the next label. Away from the imbalance boundary, either label occurs with probability 1/2, independently of the current identifier, as established in the [task dynamics section](#). This relies on  $n_{j,l} \geq 2$ , which holds for every task and label in the present model. Conditional on the new label, the identifier determines only which particular stimulus is excluded from that label class.

We can therefore group task states with the same two labels, previous task, and imbalance into a reduced state class. Summing the successor-state probabilities within each class removes their dependence on the current identifiers. The immediate reward and the probability of every reduced successor class consequently depend only on the retained state components. At the imbalance boundary, the self-loop also preserves this property. This motivates the mapping

$$\phi : S_+ \rightarrow S_\phi, s \mapsto \phi(s) := (s^{1,1}, s^{2,1}, s^3, s^4). \quad (43)$$

Its reduced state space is

$$S_\phi := \{0, 1\}^2 \times T \times B. \quad (44)$$

Thus, removing identifiers preserves both the rewards and the state-action conditional state distribution needed for planning at the level of these reduced states.

### Excluding incorrect actions

The second step restricts planning to the correct action for each task. As specified in the [task action-set section](#), an action contains a task choice and a binary response. For a given task  $j$ , the correct action  $(j, s^{j,1})$  and the incorrect action  $(j, 1 - s^{j,1})$  produce the same state-action conditional state distribution. Both affect the same task stream and have identical consequences for the previous-task component and task imbalance. They also incur the same imbalance and switching costs.

Their accuracy rewards differ by exactly one unit in favor of the correct action. Since their next-state distributions coincide, their expected future rewards are identical under any common continuation policy. The correct action therefore yields an expected total reward exactly one unit higher at every decision stage. The incorrect action is thus *strictly dominated* by the correct action for the same task, an instance of action dominance under identical transition probabilities (see Keller & Eyerich (2012), Definition 1).

Excluding these dominated actions preserves the optimal value. The two remaining choices can be represented by abstract task actions,

$$A_\chi := \{1, 2\} \quad (45)$$

where  $j$  means selecting the correct action for Task  $j$ . This abstraction retains the task choice while leaving its state-dependent response component to the action-lifting map introduced below.

### Removing stimulus labels from the planning state

Once each abstract action selects a correct response, its accuracy reward is always one, irrespective of the current stimulus labels. The imbalance and switching costs already depend only on the selected task, the previous task, and the current imbalance. These latter state components also evolve independently of the stimulus labels. The labels can therefore be removed from the planning state without changing the rewards or the probabilities of its reduced successors. This gives the regular reduced state space

$$S_{\chi,+} := T \times B \quad (46)$$

using the mapping

$$\varphi : S_\phi \rightarrow S_{\chi,+}, s \mapsto \varphi(s) := (s^3, s^4). \quad (47)$$

The intermediate state retains the component indices of the task state. The stimulus labels remain part of the task state. They are unnecessary for choosing between tasks under this reward function, but remain necessary for translating a task choice into the corresponding correct action.

Including the initial state gives the complete reduced state space

$$S_\chi := \{(\emptyset, 0)\} \cup S_{\chi,+}. \quad (48)$$

We define the combined state-reduction map on the full task state space as

$$\chi : S \rightarrow S_\chi, s \mapsto \chi(s) := (s^3, s^4). \quad (49)$$

For every regular state  $s \in S_+$ , this map satisfies  $\chi(s) = \varphi(\phi(s))$ . Every initial task state  $s \in S_0$  maps to  $(\emptyset, 0)$ , irrespective of its stimulus labels and identifiers. Thus,  $\chi$  extends the composition of the two intermediate reductions to the complete task state space. Note that  $S_{\chi,+}$  contains only regular reduced states, while  $S_\chi$  also contains the single reduced initial state.

#### *Reduced policies and action lifting*

The resulting planning problem has  $2(2n + 1)$  regular reduced states and two abstract actions. For the largest observed action count  $n = 245$ , this gives  $2(2 \cdot 245 + 1) = 982$  regular reduced states. The initial state adds one further reduced state, while the participant- and block-specific horizon remains  $n$ . For regular task states, the action probabilities  $\pi_t(s, a)$  of an unreduced stochastic policy are evaluated on  $S_+ \times A$ . Their reduced counterparts  $\pi_{\chi,t}(s, a)$  are evaluated on  $S_{\chi,+} \times A_\chi$ . For each fixed state, these probabilities sum to one over the respective action set. Including  $(\emptyset, 0)$  extends the reduced policy to the initial state. Backward induction is performed on the full reduced state space  $S_\chi$  and abstract action set  $A_\chi$ . To express a reduced task choice as an agent decision in  $D = A$ , we use the action-lifting map

$$\lambda : S \times A_\chi \rightarrow D, (s, j) \mapsto \lambda(s, j) := (j, s^{j,1}). \quad (50)$$

Note that lifting uses the full task state  $s$ , including its stimulus labels. It assigns the response component that matches the label of the selected task and returns a decision in  $D = A$ . The same lifting rule applies to every state in  $S_0$ , even though all of these states share the same reduced initial state. The resulting agent decisions select correct task actions.

### **Reduced FCDT-MDP and preservation of optimality**

The following theorem defines the unreduced and reduced decision problems and states the preservation of optimal values and optimality under action lifting.

**Theorem 2** (Reduced FCDT-MDP and preservation of optimality). *Let  $\mathcal{T}$  be the PTD-RMC task model with horizon  $n$ , and let  $\beta \in \mathbb{R}$  and  $\sigma \geq 0$ . Combining its dynamics with the subjective reward  $\rho_{\beta,\sigma}$  defines the unreduced FCDT-MDP*

$$\mathcal{M}_{\beta,\sigma} := (J, S, A, R, p(s_0), p(s_{t+1}|s_t, a_t), \rho_{\beta,\sigma}), \quad (51)$$

where

$$R := \{\rho_{\beta,\sigma}(s, a) : s \in S, a \in A\}. \quad (52)$$

Both this MDP and its reduction below have decision stages  $t = 0, \dots, n-1$ , no reward discounting, and zero remaining reward at stage  $n$ .

Let

$$S_{\chi,+} := T \times B, S_{\chi} := \{(\emptyset, 0)\} \cup S_{\chi,+}, A_{\chi} := \{1, 2\}, \quad (53)$$

and consider the state-reduction and action-lifting maps

$$\chi : S \rightarrow S_{\chi}, s \mapsto \chi(s) := (s^3, s^4), \quad (54)$$

and

$$\lambda : S \times A_{\chi} \rightarrow D, (s, a) \mapsto \lambda(s, a) := (a, s^{a,1}), \quad (55)$$

with  $D = A$ . On  $S_+$ , the map  $\chi$  equals  $\varphi \circ \phi$ . On  $S_0$ , it maps every initial task state to  $(\emptyset, 0)$ .

Define the reduced FCDT-MDP as

$$\mathcal{M}_{\chi} := (J, S_{\chi}, A_{\chi}, R_{\chi}, p_{\chi}(s_0), p_{\chi}(s_{t+1}|s_t, a_t), \rho_{\chi}), \quad (56)$$

with the following components. The dependence of  $\mathcal{M}_{\chi}$  on  $\beta$  and  $\sigma$  is left implicit.

For reduced states, we retain the coordinate indices of the task state. The initial-state distribution is defined by

$$p_{\chi}(s_0) := \begin{cases} 1 & s_0 = (\emptyset, 0), \\ 0 & \text{otherwise.} \end{cases} \quad (57)$$

The state-action conditional state distribution is defined by

$$p_{\chi}(s_{t+1}|s_t, a_t) := \begin{cases} 1 & |s_t^4| = n, s_{t+1} = s_t, \\ 1 & |s_t^4| < n, s_{t+1} = (a_t, s_t^4 - (-1)^{a_t}), \\ 0 & \text{otherwise.} \end{cases} \quad (58)$$

Thus, the reduced dynamics retain the boundary self-loop convention of the task model.

The reduced subjective reward function  $\rho_{\chi} : S_{\chi} \times A_{\chi} \rightarrow \mathbb{R}$  is defined by

$$(s, a) \mapsto \rho_{\chi}(s, a) := \begin{cases} 1 - \beta|s^4 - (-1)^a| - \sigma & s^3 \in T \setminus \{a\}, \\ 1 - \beta|s^4 - (-1)^a| & \text{otherwise,} \end{cases} \quad (59)$$

and its reward set is

$$R_{\chi} := \{\rho_{\chi}(s, a) : s \in S_{\chi}, a \in A_{\chi}\}. \quad (60)$$

Each reduced action therefore receives one unit of accuracy reward, together with the corresponding imbalance and switching costs. The initial state incurs no switching cost.

Then the reduction has the following properties.

1. *Preservation of rewards and reduced dynamics.* For every  $s \in S$ ,  $a \in A_\chi$ , and  $s' \in S_\chi$ ,

$$\rho_{\beta,\sigma}(s, \lambda(s, a)) = \rho_\chi(\chi(s), a), \quad (61)$$

and

$$\sum_{\substack{\tilde{s} \in S \\ \chi(\tilde{s}) = s'}} p(s_{t+1} = \tilde{s} | s_t = s, a_t = \lambda(s, a)) = p_\chi(s_{t+1} = s' | s_t = \chi(s), a_t = a). \quad (62)$$

Moreover, mapping the task model's initial state through  $\chi$  yields the reduced initial-state distribution specified above.

2. *Equality of optimal values.* Let  $v_t^*$  and  $v_{\chi,t}^*$  denote the optimal value functions of  $\mathcal{M}_{\beta,\sigma}$  and  $\mathcal{M}_\chi$ , respectively. Then

$$v_t^*(s) = v_{\chi,t}^*(\chi(s)) \quad (63)$$

for every  $s \in S$  and  $t \in J$ , including the initial state layer and the horizon boundary.

3. *Optimality of lifted policies.* Let  $\delta_{\chi,t}^*$  be optimal decision functions obtained by backward induction in  $\mathcal{M}_\chi$ . Then the decision functions

$$\delta_t^* : S \rightarrow D, s \mapsto \delta_t^*(s) := \lambda(s, \delta_{\chi,t}^*(\chi(s))), \quad (64)$$

for  $t = 0, \dots, n-1$ , define an optimal policy for  $\mathcal{M}_{\beta,\sigma}$ . This holds for any selection among tied optimal reduced actions and does not require the optimal policy to be unique.

■

## Proof

### Optimal policy and decisions

For fixed parameters  $\beta$  and  $\sigma$ , we derive the reduced optimal policy by backward induction in  $\mathcal{M}_\chi$  over the participant- and block-specific horizon  $n$ , without reward discounting. The construction follows Theorem 9. Let  $q_{\chi,t}^*$  denote the optimal action-value function of the reduced MDP at stage  $t$ . We select a deterministic reduced decision function by

$$\delta_{\chi,t}^* : S_\chi \rightarrow A_\chi, s \mapsto \delta_{\chi,t}^*(s) := \min \arg \max_{a \in A_\chi} q_{\chi,t}^*(s, a), \quad (65)$$

for  $t = 0, \dots, n - 1$ . When the two action values are exactly equal, this rule selects Task 1, matching the implementation described in S3. Note that this convention selects one of the possibly several optimal policies. The dependence of the decision functions on  $\beta$  and  $\sigma$  is left implicit.

To determine the agent's decision at a task state  $s$ , we first apply  $\chi$  to obtain the corresponding reduced state. The reduced decision function selects a task, and  $\lambda$  uses the stimulus labels in the full task state to assign the response component. The lifted decision function is therefore

$$\delta_t^* : S \rightarrow D, s \mapsto \delta_t^*(s) := \lambda(s, \delta_{\chi, t}^*(\chi(s))). \quad (66)$$

By Theorem 2, these lifted decision functions define an optimal policy for the unreduced internal MDP. The same construction applies to initial and regular task states.

For  $s_t \in S$  and  $d_t \in D$ , the agent's conditional decision PMF is

$$p_{\beta, \sigma}(d_t | s_t) := \mathbb{I}[d_t = \delta_t^*(s_t)], \quad (67)$$

for  $t = 0, \dots, n - 1$ . It assigns probability one to the selected optimal decision and zero to every other decision. Agent model A1 is thus deterministic conditional on the task state, decision stage, and reward parameters.

Note that  $d_t$  is an internal agent decision and is not observed by the experimenter. The equality  $D = A$  specifies that decisions and task actions use the same set of possible values. It does not identify the unobserved decision with an observed response. Their relationship will be specified in the [Data model](#).

### Agent model A0

Agent model A0 provides a parameter-free baseline with a uniformly random decision policy over  $D$ . At each stage, it selects one of the four task-response combinations independently of the task state and preceding decisions, such that

$$p(d_t | s_t) := \frac{1}{|D|} = \frac{1}{4}. \quad (68)$$

The selected decision is executed as the action, yielding a uniform action distribution. This distribution coincides with the infinite-temperature limit of the response distribution introduced below: for fixed  $\beta, \sigma$  and finite optimal action values,

$$p_{A0}(a_t | s_t) = \lim_{\tau \rightarrow \infty} p_{\beta, \sigma, \tau}(a_t | s_t) = \frac{1}{|A|} = \frac{1}{4}. \quad (69)$$

Thus, A0 describes behavior in which differences in optimal action values have no influence on response probabilities.

## Data model

We define the data model as

$$\mathcal{D} := (\mathcal{T}, \mathcal{A}_1, p_{\beta, \sigma, \tau}(a_t | s_t)), \quad (70)$$

where the task model supplies the experimental dynamics, the agent model supplies optimal action values, and  $p_{\beta, \sigma, \tau}(a_t | s_t)$  specifies the distribution of observed responses. Participant and block indices remain suppressed, and the horizon  $n$  is treated as given.

Let  $q_t^*(s, a; \beta, \sigma)$  denote the optimal action-value function of the unreduced FCDT-MDP. It combines the current subjective reward with optimal continuation over the remaining horizon. For  $s_t \in S$ ,  $a_t \in A$ , and  $t = 0, \dots, n-1$ , we define the conditional response PMF by

$$p_{\beta, \sigma, \tau}(a_t | s_t) := \frac{\exp(q_t^*(s_t, a_t; \beta, \sigma)/\tau)}{\sum_{a \in A} \exp(q_t^*(s_t, a; \beta, \sigma)/\tau)}. \quad (71)$$

The temperature parameter  $\tau > 0$  determines how strongly differences in optimal action values affect response probabilities. Smaller values concentrate probability on actions with higher values, while larger values yield a more uniform response distribution.

The action values are obtained by the ordinary backward induction underlying A1, retaining optimal continuation at each subsequent stage. The softmax specifies how these values translate into observed response probabilities without modifying the agent's planning recursion. Note that the data model therefore depends on the agent's action values, rather than solely on its selected optimal decision  $d_t$ .

## Agentic behavioral model

For one participant, let  $b = 1, \dots, m$  index the experimental blocks and let  $n_b$  denote the observed number of actions in block  $b$ . For the PTD-RMC condition,  $m = 6$ . We model the blocks as independent conditional on their shared parameter vector and fixed experimental designs. Each block starts from its own initial-state distribution, with no state carried across block boundaries. The designs and horizons are treated as given.

Let  $\theta := (\beta, \sigma, \tau)$  and write  $p_\theta(a_t | s_t) := p_{\beta, \sigma, \tau}(a_t | s_t)$ . As specified in (71), the observed action depends on the state through the complete optimal action-value function. It does not depend additionally on the optimal decision  $d_t$ . Thus, the observed action and latent decision are conditionally independent given the state:

$$p_\theta(a_t, d_t | s_t) := p_\theta(a_t | s_t) p_{\beta, \sigma}(d_t | s_t). \quad (72)$$

Equivalently,  $p_\theta(a_t | s_t, d_t) = p_\theta(a_t | s_t)$  whenever the conditioning event has positive probability.

The initial-state distribution in (24), state transition in (25), reward distribution in (33), decision distribution in (67), and action distribution in (71), together with the factorizations in (34) and (72), imply the joint distribution

$$p_\theta \left( (s_{b,0:n_b-1}, d_{b,0:n_b-1}, a_{b,0:n_b-1}, r_{b,1:n_b})_{b=1}^m \right) \\ = \prod_{b=1}^m \left( p(s_{b,0}) \prod_{t=0}^{n_b-1} p_{\beta,\sigma}(d_{b,t}|s_{b,t}) p_\theta(a_{b,t}|s_{b,t}) p(r_{b,t+1}|s_{b,t}, a_{b,t}) \prod_{t=0}^{n_b-2} p(s_{b,t+1}|s_{b,t}, a_{b,t}) \right). \quad (73)$$

This joint distribution describes the interaction between task dynamics, optimal agent decisions, and observable behavior. Note that the trial-wise rewards are latent from the perspective of the agent, while the agent's decisions are latent from the experimenters' perspective.

## Model estimation and evaluation

We estimate the parameters of the ABM by means of maximum likelihood. To this end, we first define the nature of maximum likelihood estimation for this ABM.

**Definition 1** (Sequential conditional maximum likelihood estimation). With the block structure specified in the [Agentic behavioral model](#) section, let the ABM be given by the joint distribution in (73), with parameter space

$$\Theta := \{(\beta, \sigma, \tau) \in \mathbb{R}^3 : \sigma \geq 0, \tau > 0\}, \quad (74)$$

and factor distributions specified by the task, agent, and data models above. Their known block-specific design and horizon dependence is implicit in the probability notation. For fixed observed trajectories within the task model's support, define the ABM's *sequential conditional action probability* by

$$p_\theta(a_{b,0:n_b-1} \| s_{b,0:n_b-1}, r_{b,1:n_b-1}) := \prod_{t=0}^{n_b-1} p_\theta(a_{b,t} | s_{b,0:t}, a_{b,0:t-1}, r_{b,1:t}), \quad (75)$$

for each block  $b$ , where  $\|$  denotes *sequential conditioning*, such that each action is conditioned on the current and preceding observed states, preceding observed actions, and preceding observed rewards within that block. At  $t = 0$ , the preceding-action and preceding-reward sequences are empty. Then the ABM's *sequential conditional likelihood function* is defined as

$$L : \Theta \rightarrow \mathbb{R}_{>0}, \theta \mapsto L(\theta) := \prod_{b=1}^m p_\theta(a_{b,0:n_b-1} \| s_{b,0:n_b-1}, r_{b,1:n_b-1}), \quad (76)$$

and its *sequential conditional log-likelihood function* is defined as

$$\ell : \Theta \rightarrow \mathbb{R}, \theta \mapsto \ell(\theta) := \ln L(\theta). \quad (77)$$

A *sequential conditional maximum likelihood estimate* is any

$$\hat{\theta} \in \arg \max_{\theta \in \Theta} \ell(\theta). \quad (78)$$

A rule selecting such a maximizer as a function of the observed data defines a *sequential conditional maximum likelihood estimator*.

The sequential conditional likelihood defined here is a causal likelihood in the terminology of Ziebart et al. (2013), and its factorization is related to the general partial-likelihood construction of Cox (1975). For a contemporary exposition of this construction, see Reid (2024).

**Theorem 3** (Functional form of the ABM's sequential conditional log-likelihood). *Given the definitions in Definition 1, the functional form of the sequential conditional log-likelihood is given by*

$$\begin{aligned} \ell(\theta) &= \ln \left( \prod_{b=1}^m p_{\theta}(a_{b,0:n_b-1} \| s_{b,0:n_b-1}, r_{b,1:n_b-1}) \right) \\ &= \sum_{b=1}^m \sum_{t=0}^{n_b-1} \left( q_{b,t}^*(s_{b,t}, a_{b,t}; \beta, \sigma) / \tau - \ln \left( \sum_{a \in A} \exp(q_{b,t}^*(s_{b,t}, a; \beta, \sigma) / \tau) \right) \right). \end{aligned} \quad (79)$$

Here,  $q_{b,t}^*$  denotes the optimal action-value function for the design and horizon of block  $b$ .

*Proof.* We give the following argument for a single block, for which we omit the block subscript  $b$  and write  $n$  for  $n_b$ . We first show that

$$p_{\theta}(a_t | s_{0:t}, a_{0:t-1}, r_{1:t}) = p_{\theta}(a_t | s_t) \quad (80)$$

To this end, for  $t = 0, \dots, n-1$  and observed values with  $p_{\theta}(s_{0:t}, a_{0:t-1}, r_{1:t}) > 0$ , we have

$$\begin{aligned} & p_{\theta}(a_t | s_{0:t}, a_{0:t-1}, r_{1:t}) \\ &= \frac{p_{\theta}(s_{0:t}, a_{0:t}, r_{1:t})}{p_{\theta}(s_{0:t}, a_{0:t-1}, r_{1:t})} \\ &= \frac{\sum_{d_{0:t}} p_{\theta}(s_{0:t}, d_{0:t}, a_{0:t}, r_{1:t})}{\sum_{a_t} \sum_{d_{0:t}} p_{\theta}(s_{0:t}, d_{0:t}, a_{0:t}, r_{1:t})} \\ &\stackrel{(1)}{=} \frac{\sum_{d_{0:t}} \sum_{s_{t+1:n-1}} \sum_{d_{t+1:n-1}} \sum_{a_{t+1:n-1}} \sum_{r_{t+1:n}} p_{\theta}(s_{0:n-1}, d_{0:n-1}, a_{0:n-1}, r_{1:n})}{\sum_{a_t} \sum_{d_{0:t}} \sum_{s_{t+1:n-1}} \sum_{d_{t+1:n-1}} \sum_{a_{t+1:n-1}} \sum_{r_{t+1:n}} p_{\theta}(s_{0:n-1}, d_{0:n-1}, a_{0:n-1}, r_{1:n})} \\ &\stackrel{(2)}{=} \frac{\sum_{d_{0:t}} \sum_{s_{t+1:n-1}} \sum_{d_{t+1:n-1}} \sum_{a_{t+1:n-1}} \sum_{r_{t+1:n}} p(s_0) \left( \prod_{j=0}^{n-1} p_{\beta, \sigma}(d_j | s_j) p_{\theta}(a_j | s_j) p(r_{j+1} | s_j, a_j) \right) \prod_{j=0}^{n-2} p(s_{j+1} | s_j, a_j)}{\sum_{a_t} \sum_{d_{0:t}} \sum_{s_{t+1:n-1}} \sum_{d_{t+1:n-1}} \sum_{a_{t+1:n-1}} \sum_{r_{t+1:n}} p(s_0) \left( \prod_{j=0}^{n-1} p_{\beta, \sigma}(d_j | s_j) p_{\theta}(a_j | s_j) p(r_{j+1} | s_j, a_j) \right) \prod_{j=0}^{n-2} p(s_{j+1} | s_j, a_j)} \\ &\stackrel{(3)}{=} \frac{\sum_{d_{0:t}} p(s_0) \left( \prod_{j=0}^{t-1} p_{\beta, \sigma}(d_j | s_j) p_{\theta}(a_j | s_j) p(r_{j+1} | s_j, a_j) p(s_{j+1} | s_j, a_j) \right) p_{\beta, \sigma}(d_t | s_t) p_{\theta}(a_t | s_t)}{\sum_{a_t} \sum_{d_{0:t}} p(s_0) \left( \prod_{j=0}^{t-1} p_{\beta, \sigma}(d_j | s_j) p_{\theta}(a_j | s_j) p(r_{j+1} | s_j, a_j) p(s_{j+1} | s_j, a_j) \right) p_{\beta, \sigma}(d_t | s_t) p_{\theta}(a_t | s_t)} \\ &\stackrel{(4)}{=} \frac{p(s_0) \left( \prod_{j=0}^{t-1} p(r_{j+1} | s_j, a_j) p(s_{j+1} | s_j, a_j) \right) \left( \sum_{d_{0:t-1}} \prod_{j=0}^{t-1} p_{\beta, \sigma}(d_j | s_j) p_{\theta}(a_j | s_j) \right) \sum_{d_t} p_{\beta, \sigma}(d_t | s_t) p_{\theta}(a_t | s_t)}{p(s_0) \left( \prod_{j=0}^{t-1} p(r_{j+1} | s_j, a_j) p(s_{j+1} | s_j, a_j) \right) \left( \sum_{d_{0:t-1}} \prod_{j=0}^{t-1} p_{\beta, \sigma}(d_j | s_j) p_{\theta}(a_j | s_j) \right) \sum_{a_t} \sum_{d_t} p_{\beta, \sigma}(d_t | s_t) p_{\theta}(a_t | s_t)} \\ &= \frac{\sum_{d_t} p_{\beta, \sigma}(d_t | s_t) p_{\theta}(a_t | s_t)}{\sum_{a_t} \sum_{d_t} p_{\beta, \sigma}(d_t | s_t) p_{\theta}(a_t | s_t)} \end{aligned}$$

$$\begin{aligned}
&= \frac{\sum_{d_t} p_{\beta,\sigma}(d_t|s_t) p_{\theta}(a_t|s_t)}{\sum_{d_t} p_{\beta,\sigma}(d_t|s_t) \sum_{a_t} p_{\theta}(a_t|s_t)} \\
&= \frac{\sum_{d_t} p_{\beta,\sigma}(d_t|s_t) p_{\theta}(a_t|s_t)}{\sum_{d_t} p_{\beta,\sigma}(d_t|s_t)} \\
&= \sum_{d_t} p_{\beta,\sigma}(d_t|s_t) p_{\theta}(a_t|s_t) \\
&= \sum_{d_t} p_{\theta}(d_t, a_t|s_t) \\
&= p_{\theta}(a_t|s_t).
\end{aligned}$$

Here, in (1), we express the numerator and denominator as marginals of the full ABM joint distribution. In (2), we substitute its factorization without performing any summation. In equality (3), we apply Theorem 11 to the temporal ordering

$$s_0, d_0, a_0, r_1, s_1, d_1, a_1, r_2, \dots, s_{n-1}, d_{n-1}, a_{n-1}, r_n. \quad (81)$$

Each factor is normalized with respect to its current variable for every fixed value of its preceding variables. The theorem therefore permits successive summation over all variables following  $a_t$ , leaving precisely the factors up to and including  $a_t$ . The sums over  $d_{0:t}$ , and the additional sum over  $a_t$  in the denominator, remain outside this calculation. Finally, (4) follows by exchanging finite summation and multiplication, as in Theorem S.1 of Ostwald et al. (2026), and taking factors independent of the summation variables outside the respective sums. The common factors are positive by the assumed positivity of the conditioning event and can therefore be cancelled. The remaining equalities follow from normalization, the ABM's decision and action distributions, and the product and marginalization rules.

We now restore the block index  $b$  and combine the block contributions. From Definition 1, the conditional independence in (72), and the response PMF in (71), we then have

$$\begin{aligned}
\ell(\theta) &= \ln \left( \prod_{b=1}^m p_{\theta}(a_{b,0:n_b-1} \| s_{b,0:n_b-1}, r_{b,1:n_b-1}) \right) \\
&= \ln \left( \prod_{b=1}^m \prod_{t=0}^{n_b-1} p_{\theta}(a_{b,t} | s_{b,0:t}, a_{b,0:t-1}, r_{b,1:t}) \right) \\
&= \ln \left( \prod_{b=1}^m \prod_{t=0}^{n_b-1} p_{\theta}(a_{b,t} | s_{b,t}) \right) \\
&= \sum_{b=1}^m \sum_{t=0}^{n_b-1} \ln p_{\theta}(a_{b,t} | s_{b,t}) \\
&= \sum_{b=1}^m \sum_{t=0}^{n_b-1} \ln \left( \frac{\exp(q_{b,t}^*(s_{b,t}, a_{b,t}; \beta, \sigma)/\tau)}{\sum_{a \in A} \exp(q_{b,t}^*(s_{b,t}, a; \beta, \sigma)/\tau)} \right) \\
&= \sum_{b=1}^m \sum_{t=0}^{n_b-1} \left( \ln \left( \exp(q_{b,t}^*(s_{b,t}, a_{b,t}; \beta, \sigma)/\tau) \right) - \ln \left( \sum_{a \in A} \exp(q_{b,t}^*(s_{b,t}, a; \beta, \sigma)/\tau) \right) \right) \\
&= \sum_{b=1}^m \sum_{t=0}^{n_b-1} \left( q_{b,t}^*(s_{b,t}, a_{b,t}; \beta, \sigma)/\tau - \ln \left( \sum_{a \in A} \exp(q_{b,t}^*(s_{b,t}, a; \beta, \sigma)/\tau) \right) \right).
\end{aligned}$$

□

For each participant, the parameters of A1 were estimated jointly by sequential conditional maximum likelihood using all observed actions from the six PTD-RMC blocks. Correct and incorrect responses were included. A common parameter vector  $\theta := (\beta, \sigma, \tau)$  governed all blocks of a participant, with

block-specific experimental designs and observed horizons treated as given. Writing  $\ell_p$  for participant  $p$ 's sequential conditional log-likelihood function, numerical estimation targeted

$$\hat{\theta}_p \in \arg \min_{\theta \in \Theta_{\text{num}}} (-\ell_p(\theta)), \quad (82)$$

where

$$\Theta_{\text{num}} := \{(\beta, \sigma, \tau) \in \mathbb{R}^3 : \sigma \geq 0, e^{-15} \leq \tau \leq e^{15}\}, \quad (83)$$

in an agentic log-likelihood function implementation (cf. Ostwald et al. (2026)). The numerical bounds on  $\tau$  maintain strictly positive, finite values. The parameter  $\beta$  was unrestricted in sign, and the boundary  $\sigma = 0$  was included in the optimization domain. Optimization used the adaptive Nelder–Mead simplex algorithm (Nelder & Mead (1965), Gao & Han (2012)) as implemented in `scipy.optimize.minimize`. For each participant, a single optimization run was initialized at

$$\theta^{(0)} := (0.01, 0.1, 0.3). \quad (84)$$

Convergence required both the largest absolute coordinate difference between the best simplex vertex and the remaining vertices to be at most  $10^{-8}$ , and the largest absolute objective-value difference to be at most  $10^{-9}$ . Each optimization allowed at most 2,500 iterations.

For each participant, A0 and A1 were compared using the Bayesian information criterion, expressed on the log-likelihood scale. Let

$$\nu_p := \sum_{b=1}^m n_{p,b} \quad (85)$$

denote the total number of observed actions across participant  $p$ 's blocks. For model  $M \in \{\text{A0}, \text{A1}\}$ , the comparison score was (Schwarz, 1978)

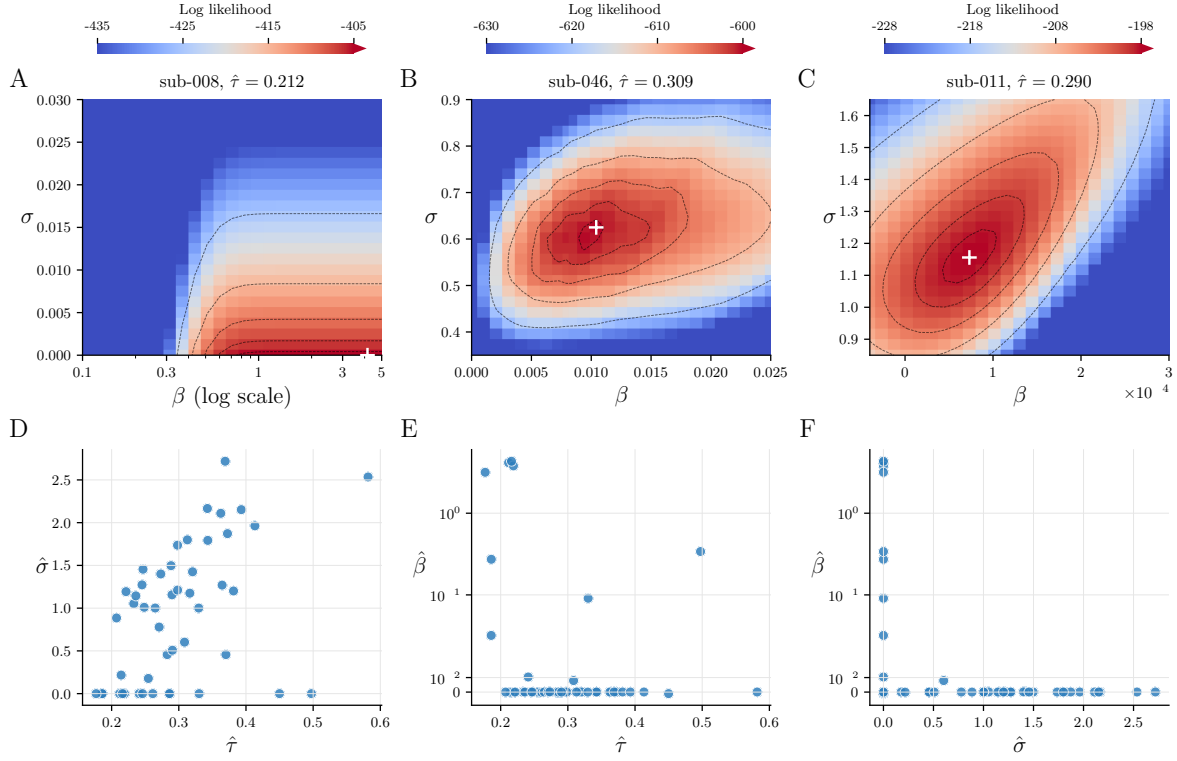
$$\text{BIC}_{p,M} := \hat{\ell}_{p,M} - \frac{k_M}{2} \ln \nu_p, \quad (86)$$

where  $\hat{\ell}_{p,M}$  denotes the attained sequential conditional log-likelihood and  $k_M$  the number of fitted parameters. Thus,  $k_{\text{A0}} = 0$  and  $k_{\text{A1}} = 3$ . The three parameters of A1 were shared across blocks, so its complexity penalty reflected three participant-level parameters. Relative support for A1 was summarized by

$$\Delta \text{BIC}_p := \text{BIC}_{p,\text{A1}} - \text{BIC}_{p,\text{A0}}. \quad (87)$$

Positive values favor A1, whereas negative values favor A0. Both models were evaluated using the same observed actions and block structure. For implementational details, please refer to *abm\_evaluation.py*.

## Preliminary results



**Figure 3.** Conditional log-likelihood surfaces and participant-specific parameter estimates for agent A1. Panels A–C show the log-likelihood as a function of  $\beta$  and  $\sigma$  for sub-008, sub-046, and sub-011, respectively. In each panel,  $\tau$  is fixed at the jointly estimated value shown above the surface. Parameter ranges and color scales are participant-specific, and Panel A uses a logarithmic  $\beta$  axis. Colors display the highest 30 log-likelihood units, with lower values saturated at the lower color limit. Contours indicate decreases of 0.5, 2, 5, 10, and 20 units from the grid maximum. White crosses mark the highest evaluated grid values. Panels D–F show the original joint estimates for all participants, with  $\hat{\tau}$  on the abscissa and  $\hat{\sigma}$  on the ordinate in Panel D,  $\hat{\tau}$  and  $\hat{\beta}$  in Panel E, and  $\hat{\sigma}$  and  $\hat{\beta}$  in Panel F. Each point represents one participant. The ordinates in Panels E and F use symmetric logarithmic scaling with a linear region near zero. For implementational details, please refer to *abm\_likelihood.py* and *abm\_figure\_3.py*.

Using the log-likelihood-scale BIC defined above, the BIC of A1 exceeded that of A0 for all 49 participants. The participant-specific differences  $\Delta\text{BIC}_p$  were therefore positive throughout, with a median of 966.11, a mean of 926.79, and a range from 475.19 for participant 009 to 1,374.70 for participant 040. Thus, BIC favored A1 for every participant despite the penalty for its three fitted parameters.

Panels A–C of Figure 3 show conditional log-likelihood surfaces in  $\beta$  and  $\sigma$ , with  $\tau$  fixed at its jointly estimated value. These surfaces illustrate different patterns of parameter sensitivity. For sub-008, the log-likelihood approaches a plateau as  $\beta$  increases at the boundary  $\sigma = 0$ , but decreases sharply as  $\sigma$

increases. Thus, the surface separates small values of  $\beta$  from sufficiently large ones, while providing little information about the magnitude of  $\beta$  within the plateau. For sub-046 and sub-011, the surfaces exhibit more localized peaks and curved ridges, along which changes in the imbalance and switching costs partially compensate for one another. For implementational details, please refer to *abm\_likelihood.py*.

Panels D–F of Figure 3 show substantial variation in the estimates across participants. Panel F includes several large positive estimates of  $\beta$  at the boundary  $\hat{\sigma} = 0$ . These estimates occur for participants with nearly balanced task allocations and frequent task switching. They are consistent with a strong estimated cost of task imbalance in the absence of a switching cost. However, the plateau illustrated for sub-008 cautions against interpreting the precise magnitude of such large estimates.

These results are to be understood as preliminary. A proper assessment of the model would require face-validity simulations that reproduce the descriptive patterns of participant behavior, as well as model-recovery and parameter-recovery simulations. These analyses are, however, only sensible once reaction times are incorporated into the model as sketched in the [Outlook](#) section.

## Outlook

The next step is to extend the present choice-based account to the temporal organization of behavior. The current model conditions on the observed number of responses within each 120-second block. It explains which task participants selected at successive decision stages, but not how their response times determined the number and temporal spacing of those decisions. A more complete model should treat both the selected response and the time at which it is made as outcomes of the agent’s policy. This would allow the analysis to address the central performance objective of the FCDT paradigm: using the limited block time efficiently to produce as many correct responses as possible.

A natural framework for this extension is a finite-time semi-Markov decision process. Decisions would occur at random epochs, and each action would jointly determine a distribution over the duration until the next response and the successor task state. The corresponding semi-Markov kernel would therefore describe both what happens next and how much of the 120-second horizon is consumed. The agent’s effective control state would include the physical task state and the remaining time. Its policy could then optimize the task-response decision together with a waiting-time decision. Responding sooner would preserve time for later trials, whereas waiting could improve preparation or response accuracy. The resulting optimality equation would be a renewal-type Bellman equation indexed by remaining calendar time rather than the finite-stage recursion used in the present model. This development can draw on the standard semi-MDP framework and its finite-horizon extensions (Ascione & Cuomo, 2022; Huang & Guo, 2011; Jewell, 1963; Puterman, 1994, Chapter 11; Tijms, 2003, Chapter 7).

Several technical points from this literature will be important. The model must specify what happens when a response would occur after the block deadline and whether a response exactly at the deadline

receives a reward. Nonexplosion and integrability conditions are required to ensure that the expected total reward is well defined. If response times are represented on a discrete temporal grid with strictly positive durations, the problem can be solved by backward induction on the remaining time. For continuous response times, the value function becomes a function of continuous remaining time. Numerical implementation would then require a time grid, quadrature, or an approximation of the duration distributions by discrete distributions. The state-space reductions used in the present report would also have to be reconsidered because response times and accuracy may depend on the currently displayed stimuli or on their processing history.

The temporal extension should be accompanied by a cognitively richer account of subjective reward. The present reward function represents correct responding, task balance, and a fixed cost of switching. This provides a useful first approximation, but it does not directly represent the processing demands that the experiment was designed to manipulate (Brüning et al., 2020). A future reward function could include costs of perceptual processing, response preparation, and maintaining or updating task representations. These costs need not be constant across tasks or trials. They could depend on stimulus properties, the time devoted to processing, the amount of preparation already completed, and whether processing resources are shared by the two tasks.

Importantly, frequent switching need not be represented solely as costly. In the FCDDT paradigm, both stimuli remain continuously visible, so the stimulus of the unselected task can be previewed while the participant prepares or executes a response to the selected task. This preview may reduce the time or processing effort required for a subsequent response. Frequent alternation could therefore provide an efficiency benefit when preparation of the alternative task is possible, even if switching itself carries a cost. The same mechanism may help explain response grouping, where a longer processing interval is followed by two closely spaced responses. A richer state representation could track the preparation or accumulated evidence associated with each task stream, allowing the model to express the trade-off between acting immediately, waiting for further processing, repeating the current task, and switching to a previewed task.

This extension is ambitious but builds directly on the present framework. The current finite-horizon MDP establishes the task dynamics, the relation between subjective costs and task choices, and the likelihood-based connection to participant behavior. Adding semi-Markov timing and processing variables would make it possible to evaluate response choice, response time, and task efficiency within a common optimization model. Once this extension has been specified and implemented, its face validity should be examined through simulations of the principal behavioral patterns, followed by model-recovery and parameter-recovery analyses. These steps would provide a more informative basis for assessing whether individual FCDDT strategies reflect different subjective costs, different processing constraints, or different uses of stimulus preview and concurrent preparation under the rational analysis framework.

## References

- Arrow, K. J., Blackwell, D., & Girshick, M. A. (1949). Bayes and Minimax Solutions of Sequential Decision Problems. *Econometrica*, 17(3/4), 213. <https://doi.org/10.2307/1905525>
- Ascione, G., & Cuomo, S. (2022). A Sojourn-Based Approach to Semi-Markov Reinforcement Learning. *Journal of Scientific Computing*, 92(2), 36. <https://doi.org/10.1007/s10915-022-01876-x>
- Bellman, R. (1954). The theory of dynamic programming. *Bulletin of the American Mathematical Society*, 60(6), 503–515. <https://doi.org/10.1090/S0002-9904-1954-09848-8>
- Bellman, R. (1957). A Markovian Decision Process. *Indiana University Mathematics Journal*, 6(4), 679–684. <https://doi.org/10.1512/iumj.1957.6.56038>
- Bertsekas, D. P. (2005). *Dynamic Programming and Optimal Control* (3rd ed., Vol. 1). Athena Scientific.
- Brüning, J., Mückstein, M., & Manzey, D. (2020). Multitasking strategies make the difference: Separating processing-code resources boosts multitasking efficiency when individuals prefer to interleave tasks in free concurrent dual tasking. *Journal of Experimental Psychology: Human Perception and Performance*, 46(12), 1411–1433. <https://doi.org/10.1037/xhp0000865>
- Brüning, J., Ullsperger, M., & Ostwald, D. (2025). *Individual determinants of response strategy formation in voluntary task switching*.
- Cox, D. R. (1975). Partial likelihood. *Biometrika*, 62(2), 269–276. <https://doi.org/10.1093/biomet/62.2.269>
- Drugowitsch, J., Moreno-Bote, R., Churchland, A. K., Shadlen, M. N., & Pouget, A. (2012). The Cost of Accumulating Evidence in Perceptual Decision Making. *Journal of Neuroscience*, 32(11), 3612–3628. <https://doi.org/10.1523/JNEUROSCI.4010-11.2012>
- Gao, F., & Han, L. (2012). Implementing the Nelder-Mead simplex algorithm with adaptive parameters. *Computational Optimization and Applications*, 51(1), 259–277. <https://doi.org/10.1007/s10589-010-9329-3>
- Huang, Y., & Guo, X. (2011). Finite horizon semi-Markov decision processes with application to maintenance systems. *European Journal of Operational Research*, 212(1), 131–140. <https://doi.org/10.1016/j.ejor.2011.01.027>
- Jewell, W. S. (1963). Markov-Renewal Programming. I: Formulation, Finite Return Models. *Operations Research*, 11(6), 938–948. <https://www.jstor.org/stable/167834>
- Keller, T., & Eyerich, P. (2012). PROST: Probabilistic Planning Based on UCT. *Proceedings of the Twenty-Second International Conference on Automated Planning and Scheduling*, 119–127. <https://doi.org/10.1609/icaps.v22i1.13518>
- Kochenderfer, M. J., Wheeler, T. A., & Wray, K. H. (2022). *Algorithms for decision making*. The MIT Press.
- Krishnamurthy, V. (2025). *Partially Observed Markov Decision Processes: Filtering, Learning and Controlled Sensing* (2nd ed.). Cambridge University Press. <https://doi.org/10.1017/9781009449441>
- Nelder, J. A., & Mead, R. (1965). A Simplex Method for Function Minimization. *The Computer Journal*, 7(4), 308–313. <https://doi.org/10.1093/comjnl/7.4.308>

- Ostwald, D., Bruckner, R., Usée, F., Fleischmann, B., Soch, J., & Mulready, S. (2026). *On agentic behavioral modeling*. <https://doi.org/10.48550/arXiv.2604.27894>
- Powell, W. B. (2022). *Reinforcement learning and stochastic optimization: A unified framework for sequential decisions*. Wiley.
- Puterman, M. (1994). *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons.
- Ramachandran, D., & Amir, E. (2007). Bayesian Inverse Reinforcement Learning. In M. M. Veloso (Ed.), *Proceedings of the 20th International Joint Conference on Artificial Intelligence* (pp. 2586–2591). AAAI Press / International Joint Conferences on Artificial Intelligence.
- Rao, A., & Jelvis, T. (2022). *Foundations of Reinforcement Learning with Applications in Finance* (1st ed.). Chapman and Hall/CRC. <https://doi.org/10.1201/9781003229193>
- Reid, N. (2024). On partial likelihood. *Journal of the Royal Statistical Society Series A: Statistics in Society*, 187(3), 567–577. <https://doi.org/10.1093/jrssa/qnae008>
- Schwarz, G. (1978). Estimating the Dimension of a Model. *The Annals of Statistics*, 6(2), 461–464.
- Tijms, H. C. (2003). *A First Course in Stochastic Models* (1st ed.). Wiley. <https://doi.org/10.1002/047001363X>
- Ziebart, B. D., Bagnell, J. A., & Dey, A. K. (2013). The Principle of Maximum Causal Entropy for Estimating Interacting Processes. *IEEE Transactions on Information Theory*, 59(4), 1966–1980. <https://doi.org/10.1109/TIT.2012.2234824>

## Supplementary information

### S1 Finite dynamic programming

In these notes, we review the derivation of optimal policies for Markov decision processes with finite time horizon and finite state, action, and reward spaces. We make the additional assumption of no terminal states. We first establish the basic terminology by defining Markov decision processes, decision functions and policies, and policy-dependent Markov decision processes. We then establish conditional independence properties and introduce the return, value function, and action-value function, together with their recursive relationships. Finally, after defining optimal value functions and optimal policies, we prove the central result in Theorem 9, which shows how a deterministic Markovian optimal policy can be constructed algorithmically by backward induction. The material covered here is discussed in Puterman (1994, Chapter 2 and 4), Bertsekas (2005, Volume I, Chapter 1), Powell (2022, Chapter 14), Krishnamurthy (2025, Chapter 7), and Rao & Jelvis (2022, Chapter 4 and 5).

#### Markov decision processes

**Definition 2** (Finite-horizon Markov decision process). A discrete-time, finite-horizon, time-homogeneous *Markov decision process* is a tuple

$$\text{MDP} = (J, S, A, R, \mathcal{V}, p(s_0), p(r_{t+1}, s_{t+1} | s_t, a_t), p(s_{t+1} | s_t, a_t), \rho), \quad (88)$$

whose components satisfy the following conditions:

- *Time indices.* For a fixed horizon  $T \in \mathbb{N}$ ,  $J := \{0, 1, \dots, T\}$  is the discrete time-index set. Actions are selected at the decision stages  $t = 0, 1, \dots, T-1$ . Stage  $T$  is the horizon boundary.
- *State space.*  $S$  is a finite set of state values.
- *Action set.*  $A$  is a nonempty finite set of action values. Every action in  $A$  is available in every state at every decision stage.
- *Reward set.*  $R \subseteq \mathbb{R}$  is a finite set of reward values. Finiteness of  $R$  ensures that every conditional reward expectation below exists and is finite.
- *Random-variable sequence.*

$$\mathcal{V} := (s_0, a_0, r_1, s_1, a_1, r_2, s_2, \dots, a_{T-1}, r_T, s_T) \quad (89)$$

is a finite sequence of random variables. This sequence describes the interaction between the agent and the environment. The initial state is generated according to  $p(s_0)$ . At each decision

stage, after the agent selects an action, the next reward and state are generated according to the prescribed transition model. A policy, introduced below, specifies action selection and thereby determines the joint distribution of the complete sequence. For every  $t \in J$ ,  $s_t$  takes values in  $S$ . For every  $t \in \{0, 1, \dots, T-1\}$ ,  $a_t$  takes values in  $A$ , and  $r_{t+1}$  takes values in  $R$ .

- *Initial-state distribution.*  $p(s_0)$  specifies the distribution of  $s_0$  on  $S$ .
- *State- and action-conditional state-reward probabilities.*  $p(r_{t+1}, s_{t+1} | s_t, a_t)$  denotes the collection of joint conditional probabilities for all  $t \in \{0, 1, \dots, T-1\}$ , current states in  $S$ , actions in  $A$ , next states in  $S$ , and rewards in  $R$ . For each current state, action, and time, they specify a probability distribution over the next reward and next state. These probabilities are prescribed for every listed current state and action and thereby specify a version even at conditioning values having probability zero. They satisfy the *conditional independence property*

$$p(r_{t+1}, s_{t+1} | s_{0:t}, a_{0:t}) = p(r_{t+1}, s_{t+1} | s_t, a_t) \quad (90)$$

and *time homogeneity*. Here, corresponding current-state, action, next-state, and reward variables on both sides are evaluated at identical values. For all  $t, k \in \{0, 1, \dots, T-1\}$ ,

$$p(r_{t+1}, s_{t+1} | s_t, a_t) = p(r_{k+1}, s_{k+1} | s_k, a_k). \quad (91)$$

- *State- and action-conditional state probabilities.*  $p(s_{t+1} | s_t, a_t)$  denotes the collection of conditional next-state probabilities obtained by marginalizing over rewards,

$$p(s_{t+1} | s_t, a_t) = \sum_{r_{t+1}} p(r_{t+1}, s_{t+1} | s_t, a_t), \quad (92)$$

referred to as *induced state-transition probabilities*. By marginalization, both the conditional independence property of the state- and action-conditional state-reward probabilities and their time homogeneity also apply to the state- and action-conditional state probabilities, i.e.,

$$p(s_{t+1} | s_{0:t}, a_{0:t}) = p(s_{t+1} | s_t, a_t) \quad (93)$$

and, for all  $t, k \in \{0, 1, \dots, T-1\}$ , with the same matching-value convention,

$$p(s_{t+1} | s_t, a_t) = p(s_{k+1} | s_k, a_k). \quad (94)$$

- *Action-conditional reward function.*  $\rho$  is defined as

$$\rho : S \times A \rightarrow \mathbb{R}, (s, a) \mapsto \rho(s, a) := \mathbb{E}(r_{t+1} | s_t = s, a_t = a), \quad (95)$$

where

$$\mathbb{E}(r_{t+1}|s_t, a_t) = \sum_{r_{t+1}} \sum_{s_{t+1}} r_{t+1} p(r_{t+1}, s_{t+1}|s_t, a_t). \quad (96)$$

•

Note that although we use the term *finite-horizon MDP*, the definition imposes finiteness not only on the time horizon but also on the state, action, and reward spaces. For an externally supplied action sequence  $a_{0:T-1}$ , the initial-state distribution and the state- and action-conditional state-reward probabilities determine the joint distribution of the state and reward sequences, which factorizes as

$$p(s_{0:T}, r_{1:T}|a_{0:T-1}) = p(s_0) \prod_{t=0}^{T-1} p(r_{t+1}, s_{t+1}|s_t, a_t). \quad (97)$$

Note that here, conditioning on  $a_{0:T-1}$  denotes prescribing the action sequence externally, rather than conditioning on actions generated by a state-dependent policy.

The MDP specifies how rewards and states evolve given the selected actions. We next define decision functions and policies, which specify how these actions are selected.

**Definition 3** (Decision functions and policies). For a finite-horizon Markov decision process, let  $J := \{0, 1, \dots, T\}$ , let  $S$  be its finite state space, and let  $A$  be its nonempty finite action set. Then the following definitions apply.

- *Histories*. For every  $t \in J$ , the *history at time  $t$*  is represented by

$$h_t := (s_0, a_0, s_1, a_1, \dots, s_{t-1}, a_{t-1}, s_t). \quad (98)$$

We assume that the agent observes states and its own actions, but does not observe rewards. Past rewards are therefore excluded from its history. At  $t = 0$ , the history consists only of  $s_0$ . At a decision stage  $t < T$ , this is the information available when choosing action  $a_t$ . At  $t = T$ , it is the terminal history. We also write a history as  $(s_{0:t}, a_{0:t-1})$ , grouping its state and action components.

A history is called *admissible* if it has positive probability under at least one policy, given the initial-state distribution and transition model of the MDP. For state values in  $S$  and action values in  $A$ , this is equivalent to

$$p(s_0) \prod_{j=0}^{t-1} p(s_{j+1}|s_j, a_j) > 0. \quad (99)$$

The empty product at  $t = 0$  equals one. Let  $H_t$  denote the set of these admissible histories. Admissibility does not require positive probability under the particular policy being evaluated.

- *Stochastic decision functions.* A stochastic decision function at time  $t$  specifies a probability distribution over  $A$  for each history,

$$\pi_t : H_t \times A \rightarrow [0, 1], \quad (100)$$

where  $H_t$  denotes the set of admissible histories ending in a state at time  $t$ . For every  $h \in H_t$ , the stochastic decision function satisfies

$$\sum_a \pi_t(h, a) = 1. \quad (101)$$

These action probabilities are prescribed for every admissible history.

- *Decision functions.* A decision function at time  $t$  selects one action for each history,

$$d_t : H_t \rightarrow A. \quad (102)$$

- *Stochastic policies.* A stochastic policy is a sequence of stochastic decision functions,

$$\pi := (\pi_t)_{t=0}^{T-1}. \quad (103)$$

Thus,  $\pi_t$  specifies how an action is selected at one time, whereas  $\pi$  specifies action selection at all decision stages. Together with the initial-state distribution and the action-conditional transition model,  $\pi$  induces a probability distribution  $p^\pi$  for the complete process. By construction,

$$p^\pi(a_t|h_t) = \pi_t(h_t, a_t) \quad (104)$$

for every  $t = 0, 1, \dots, T-1$ ,  $h_t \in H_t$ , and  $a_t \in A$ .

For a policy  $\pi$  and an admissible history, a *continuation expectation* of a future random variable is evaluated by fixing that history and generating subsequent actions according to  $\pi$  and subsequent rewards and states according to the MDP transition model. This convention also applies when the fixed history has probability zero under  $\pi$ . At histories of positive probability, it agrees with the ordinary conditional expectation. Conditional action probabilities at the fixed history are the probabilities prescribed by the policy there.

- *Deterministic policies.* A deterministic policy is a sequence of decision functions

$$d := (d_t)_{t=0}^{T-1}. \quad (105)$$

It can equivalently be represented as a stochastic policy whose stochastic decision functions satisfy

$$\pi_t(h, a) = \begin{cases} 1 & a = d_t(h), \\ 0 & a \neq d_t(h). \end{cases} \quad (106)$$

Thus, deterministic policies are special cases of stochastic policies.

- *Markovian and non-Markovian policies.* A stochastic or deterministic policy is *Markovian* if its time-indexed action-selection rules depend on a history only through its current state. For a stochastic policy, its stochastic decision functions can then be represented as

$$\pi_t : S \times A \rightarrow [0, 1], \quad (107)$$

where the same symbol is used for the state-based representation, so that

$$\pi_t(h, a) = \pi_t(s, a) \quad (108)$$

whenever the history  $h \in H_t$  ends in state  $s \in S$ . For a deterministic policy, its decision functions can be represented as

$$d_t : S \rightarrow A, \quad (109)$$

with  $d_t(h) = d_t(s)$  whenever  $h$  ends in  $s$ .

A Markovian policy specifies its action probabilities at all states and decision stages, including states that end no admissible history. These probabilities sum to one over  $A$  at each state and decision stage. Their specification is part of the policy and determines its value when starting from such a state. Likewise, a deterministic Markovian policy specifies an action at every state and decision stage.

A policy is *non-Markovian*, or *history-dependent*, if its action selection can differ between histories having the same current state at the same time.

- *Stationary and nonstationary Markovian policies.* A Markovian stochastic policy is *stationary* if, for all  $t, l \in \{0, 1, \dots, T-1\}$ ,  $s \in S$ , and  $a \in A$ ,

$$\pi_t(s, a) = \pi_l(s, a). \quad (110)$$

We then write  $\pi(s, a)$  for this common stochastic decision function. A Markovian deterministic policy is *stationary* if, for all  $t, l \in \{0, 1, \dots, T-1\}$  and  $s \in S$ ,

$$d_t(s) = d_l(s). \quad (111)$$

We then write  $d(s)$  for the common decision function. Markovian policies whose time-indexed action-selection rules vary with time are *nonstationary*.

Note that although the transition model and reward function are time homogeneous, a finite-horizon policy need not be stationary. Because the number of remaining decision stages depends on the current time, an optimal policy may use different decision functions at different times. Stationary policies therefore form a special subclass of the policies considered here. Combining an MDP with a Markovian stochastic policy yields the construct of a *policy-dependent MDP*, as defined next.

•

**Definition 4** (Finite-horizon policy-dependent Markov decision process). A discrete-time, finite-horizon *policy-dependent Markov decision process* under  $\pi$  is a tuple

$$\text{MDP}^\pi := (\text{MDP}, \pi, \mathcal{V}^\pi, p(s_0), p^\pi(r_{t+1}, s_{t+1}|s_t), p^\pi(s_{t+1}|s_t)), \quad (112)$$

whose components satisfy the following conditions:

- *Markov decision process.* MDP is a finite-horizon, time-homogeneous Markov decision process as in Definition 2.
- *Stochastic policy.*  $\pi := (\pi_t)_{t=0}^{T-1}$  is a Markovian stochastic policy as defined in Definition 3. The conditional version specified by the policy satisfies

$$\pi_t(s_t, a_t) = p^\pi(a_t|s_t) \quad (113)$$

for every  $t = 0, 1, \dots, T-1$ ,  $s_t \in S$ , and  $a_t \in A$ .

Note that while the superscript  $\pi$  denotes the complete policy, the conditional action probabilities are determined by the decision function  $\pi_t$  at the respective time. These decision functions may differ across times. Consequently, even when corresponding state and action variables are evaluated at identical values,

$$p^\pi(a_t|s_t) = p^\pi(a_k|s_k) \quad (114)$$

need not hold for  $t \neq k$ . For a stationary policy, this equality holds for all decision stages.

- *Policy-dependent random-variable sequence.*

$$\mathcal{V}^\pi := (s_0, a_0, r_1, s_1, a_1, r_2, s_2, \dots, a_{T-1}, r_T, s_T) \quad (115)$$

is a finite sequence of random variables on the common probability space induced by  $p(s_0)$ ,  $\pi$ , and the action-conditional transition model of MDP. At each decision stage  $t$ , the agent selects  $a_t$  according to  $\pi_t$ , after which the environment generates  $r_{t+1}$  and  $s_{t+1}$ .

- *Initial-state distribution.*  $p(s_0)$  specifies the distribution of  $s_0$  on  $S$ .

- *Policy-dependent state-conditional state-reward probabilities.* For every  $t = 0, 1, \dots, T - 1$ , the policy-dependent state-conditional state-reward probabilities are defined as

$$p^\pi(r_{t+1}, s_{t+1} | s_t) := \sum_{a_t} \pi_t(s_t, a_t) p(r_{t+1}, s_{t+1} | s_t, a_t). \quad (116)$$

Because the policy is Markovian and the transition model satisfies the conditional independence property,

$$p^\pi(r_{t+1}, s_{t+1} | s_{0:t}) = p^\pi(r_{t+1}, s_{t+1} | s_t). \quad (117)$$

- *Policy-dependent state-conditional state probabilities.* For every  $t = 0, 1, \dots, T - 1$ , the policy-dependent state-conditional state probabilities are defined as

$$p^\pi(s_{t+1} | s_t) := \sum_{r_{t+1}} p^\pi(r_{t+1}, s_{t+1} | s_t). \quad (118)$$

By marginalization, they satisfy

$$p^\pi(s_{t+1} | s_{0:t}) = p^\pi(s_{t+1} | s_t). \quad (119)$$

•

For a given Markovian stochastic policy  $\pi$ , the initial-state distribution, the policy, and the state- and action-conditional state-reward probabilities determine the joint distribution of the state, action, and reward sequences. This distribution is parameterized by  $\pi$  and factorizes as

$$p^\pi(s_{0:T}, a_{0:T-1}, r_{1:T}) = p(s_0) \prod_{t=0}^{T-1} p^\pi(a_t | s_t) p(r_{t+1}, s_{t+1} | s_t, a_t). \quad (120)$$

Note that although the underlying MDP is time homogeneous, the policy-dependent transition probabilities  $p^\pi(r_{t+1}, s_{t+1} | s_t)$  and  $p^\pi(s_{t+1} | s_t)$  are generally not time homogeneous. In particular, their definitions combine the time-homogeneous MDP transition model with the time-indexed action probabilities  $p^\pi(a_t | s_t) = \pi_t(s_t, a_t)$ . Consequently,  $p^\pi(r_{t+1}, s_{t+1} | s_t)$  and  $p^\pi(s_{t+1} | s_t)$  may vary with  $t$  whenever the policy is nonstationary. If  $\pi$  is stationary, the time homogeneity of the underlying MDP implies that  $p^\pi(r_{t+1}, s_{t+1} | s_t)$  and  $p^\pi(s_{t+1} | s_t)$  are time homogeneous.

In the following theorem, we establish a conditional independence property of future rewards that allows us to show that the conditional expected return from time  $t + 1$  onward depends only on  $s_{t+1}$ , with no additional dependence on  $s_t$  and  $a_t$  (Theorem 6). This result in turn supports the recursive relationship between the action-value function at time  $t$  and the value function at time  $t + 1$  (Theorem 7).

**Theorem 4** (Reward conditional independence for a policy-dependent MDP). *Let  $\pi$  be the Markovian stochastic policy of a policy-dependent MDP. Then, for every  $t = 0, 1, \dots, T - 2$ ,  $k = t + 2, t + 3, \dots, T$ , and every state-action-state combination  $(s_t, a_t, s_{t+1}) \in S \times A \times S$  with  $p^\pi(s_t, a_t, s_{t+1}) > 0$ , the reward at time  $k$  is conditionally independent of the state and action at time  $t$  given the state at time  $t + 1$ ,*

$$p^\pi(r_k | s_t, a_t, s_{t+1}) = p^\pi(r_k | s_{t+1}). \quad (121)$$

■

*Proof.* We first prove the stronger identity

$$p^\pi(r_k | s_{0:t+1}, a_{0:t}) = p^\pi(r_k | s_{t+1}) \quad (122)$$

for every positive-probability state-action history  $(s_{0:t+1}, a_{0:t})$ . All sums below involving conditional probabilities are restricted to conditioning events of positive probability. Omitted events have zero weight.

To this end, we use induction on

$$d := k - t - 1, \quad (123)$$

i.e., the number of time steps by which the reward  $r_k$  lies beyond  $s_{t+1}$ .

*Base case.* Let  $d = 1$ , so that  $k = t + 2$ . Then we have

$$\begin{aligned} p^\pi(r_{t+2} | s_{0:t+1}, a_{0:t}) &= \sum_{a_{t+1}} \sum_{s_{t+2}} p^\pi(a_{t+1}, r_{t+2}, s_{t+2} | s_{0:t+1}, a_{0:t}) \\ &= \sum_{a_{t+1}} \sum_{s_{t+2}} p^\pi(r_{t+2}, s_{t+2} | a_{t+1}, s_{0:t+1}, a_{0:t}) p^\pi(a_{t+1} | s_{0:t+1}, a_{0:t}) \\ &= \sum_{a_{t+1}} \sum_{s_{t+2}} p^\pi(a_{t+1} | s_{0:t+1}, a_{0:t}) p^\pi(r_{t+2}, s_{t+2} | s_{0:t+1}, a_{0:t+1}) \\ &\stackrel{(1)}{=} \sum_{a_{t+1}} \sum_{s_{t+2}} p^\pi(a_{t+1} | s_{t+1}) p(r_{t+2}, s_{t+2} | s_{t+1}, a_{t+1}) \\ &= \sum_{a_{t+1}} \sum_{s_{t+2}} \pi_{t+1}(s_{t+1}, a_{t+1}) p(r_{t+2}, s_{t+2} | s_{t+1}, a_{t+1}) \\ &= p^\pi(r_{t+2} | s_{t+1}). \end{aligned}$$

The second equality applies the chain rule. The third equality reorders the factors and combines the conditioning actions into  $a_{0:t+1}$ . Equality (1) applies the Markovian property of  $\pi$  and the state- and action-conditional independence property of the MDP. The subsequent equality uses  $p^\pi(a_{t+1} | s_{t+1}) = \pi_{t+1}(s_{t+1}, a_{t+1})$ .

*Induction step.* Let  $d \geq 2$ , and assume that the induction hypothesis in (122) holds for every positive-probability state-action history whose final conditioning state has distance  $d - 1$  from the reward. With  $s_{t+2}$  as the final conditioning state and  $r_k$  as the reward, the induction hypothesis then takes the form

$$p^\pi(r_k | s_{0:t+2}, a_{0:t+1}) = p^\pi(r_k | s_{t+2}). \quad (124)$$

Here, the relevant distance is indeed  $d - 1$ , because

$$k - (t + 1) - 1 = k - t - 2 = (k - t - 1) - 1 = d - 1.$$

We then have

$$\begin{aligned}
& p^\pi(r_k | s_{0:t+1}, a_{0:t}) \\
&= \sum_{a_{t+1}} \sum_{s_{t+2}} p^\pi(r_k, a_{t+1}, s_{t+2} | s_{0:t+1}, a_{0:t}) \\
&= \sum_{a_{t+1}} \sum_{s_{t+2}} p^\pi(r_k | s_{0:t+2}, a_{0:t+1}) p^\pi(a_{t+1}, s_{t+2} | s_{0:t+1}, a_{0:t}) \\
&\stackrel{(2)}{=} \sum_{a_{t+1}} \sum_{s_{t+2}} p^\pi(r_k | s_{t+2}) p^\pi(a_{t+1}, s_{t+2} | s_{0:t+1}, a_{0:t}) \\
&= \sum_{a_{t+1}} \sum_{s_{t+2}} p^\pi(r_k | s_{t+2}) p^\pi(s_{t+2} | a_{t+1}, s_{0:t+1}, a_{0:t}) p^\pi(a_{t+1} | s_{0:t+1}, a_{0:t}) \\
&= \sum_{a_{t+1}} \sum_{s_{t+2}} p^\pi(r_k | s_{t+2}) p^\pi(a_{t+1} | s_{0:t+1}, a_{0:t}) p^\pi(s_{t+2} | s_{0:t+1}, a_{0:t}, a_{t+1}) \\
&= \sum_{a_{t+1}} \sum_{s_{t+2}} p^\pi(r_k | s_{t+2}) p^\pi(a_{t+1} | s_{0:t+1}, a_{0:t}) p^\pi(s_{t+2} | s_{0:t+1}, a_{0:t+1}) \\
&\stackrel{(3)}{=} \sum_{a_{t+1}} \sum_{s_{t+2}} p^\pi(r_k | s_{t+2}) p^\pi(a_{t+1} | s_{t+1}) p(s_{t+2} | s_{t+1}, a_{t+1}) \\
&= \sum_{a_{t+1}} \sum_{s_{t+2}} p^\pi(r_k | s_{t+2}) \pi_{t+1}(s_{t+1}, a_{t+1}) p(s_{t+2} | s_{t+1}, a_{t+1}) \\
&= \sum_{a_{t+1}} \sum_{s_{t+2}} p^\pi(r_k | s_{t+2}) \left( \sum_{s_{0:t}} \sum_{a_{0:t}} p^\pi(s_{0:t}, a_{0:t} | s_{t+1}, a_{t+1}, s_{t+2}) \right) \pi_{t+1}(s_{t+1}, a_{t+1}) p(s_{t+2} | s_{t+1}, a_{t+1}) \\
&= \sum_{a_{t+1}} \sum_{s_{t+2}} \left( \sum_{s_{0:t}} \sum_{a_{0:t}} p^\pi(r_k | s_{t+2}) p^\pi(s_{0:t}, a_{0:t} | s_{t+1}, a_{t+1}, s_{t+2}) \right) \pi_{t+1}(s_{t+1}, a_{t+1}) p(s_{t+2} | s_{t+1}, a_{t+1}) \\
&\stackrel{(4)}{=} \sum_{a_{t+1}} \sum_{s_{t+2}} \left( \sum_{s_{0:t}} \sum_{a_{0:t}} p^\pi(r_k | s_{0:t+2}, a_{0:t+1}) p^\pi(s_{0:t}, a_{0:t} | s_{t+1}, a_{t+1}, s_{t+2}) \right) \pi_{t+1}(s_{t+1}, a_{t+1}) p(s_{t+2} | s_{t+1}, a_{t+1}) \\
&= \sum_{a_{t+1}} \sum_{s_{t+2}} \left( \sum_{s_{0:t}} \sum_{a_{0:t}} p^\pi(r_k, s_{0:t}, a_{0:t} | s_{t+1}, a_{t+1}, s_{t+2}) \right) \pi_{t+1}(s_{t+1}, a_{t+1}) p(s_{t+2} | s_{t+1}, a_{t+1}) \\
&= \sum_{a_{t+1}} \sum_{s_{t+2}} p^\pi(r_k | s_{t+1}, a_{t+1}, s_{t+2}) \pi_{t+1}(s_{t+1}, a_{t+1}) p(s_{t+2} | s_{t+1}, a_{t+1}) \\
&= \sum_{a_{t+1}} \sum_{s_{t+2}} p^\pi(r_k | s_{t+1}, a_{t+1}, s_{t+2}) p^\pi(a_{t+1}, s_{t+2} | s_{t+1}) \\
&= \sum_{a_{t+1}} \sum_{s_{t+2}} p^\pi(r_k, a_{t+1}, s_{t+2} | s_{t+1}) \\
&= p^\pi(r_k | s_{t+1}). \tag{125}
\end{aligned}$$

Here, (2) applies the induction hypothesis, (3) applies the Markovian property of the policy and the conditional independence of the next state in the MDP and (4) applies the induction hypothesis in the reverse direction. This proves (122) for distance  $d$ .

It remains to deduce the theorem's assertion from (122). To this end, we have

$$\begin{aligned}
p^\pi(r_k | s_t, a_t, s_{t+1}) &= \sum_{s_{0:t-1}} \sum_{a_{0:t-1}} p^\pi(r_k, s_{0:t-1}, a_{0:t-1} | s_t, a_t, s_{t+1}) \\
&= \sum_{s_{0:t-1}} \sum_{a_{0:t-1}} p^\pi(r_k | s_{0:t+1}, a_{0:t}) p^\pi(s_{0:t-1}, a_{0:t-1} | s_t, a_t, s_{t+1}) \\
&= \sum_{s_{0:t-1}} \sum_{a_{0:t-1}} p^\pi(r_k | s_{t+1}) p^\pi(s_{0:t-1}, a_{0:t-1} | s_t, a_t, s_{t+1})
\end{aligned}$$

$$\begin{aligned}
&= p^\pi(r_k | s_{t+1}) \sum_{s_{0:t-1}} \sum_{a_{0:t-1}} p^\pi(s_{0:t-1}, a_{0:t-1} | s_t, a_t, s_{t+1}) \\
&= p^\pi(r_k | s_{t+1})
\end{aligned} \tag{126}$$

At  $t = 0$ , the preceding state-action history is empty and its conditional probability is one.

□

### Policy-dependent value and action-value functions

The fundamental goal of policy optimization is to maximize the expected cumulative reward over the remaining decision stages, which may comprise all decision stages when starting at the initial time. In MDP theory, this objective is formalized through the return, the value function, and the action-value function. The return is the sum of future rewards, optionally discounted to give less weight to rewards received later. The value function gives the expected return when starting from a given state and following a policy, whereas the action-value function gives the expected return when additionally prescribing the first action. Together, these quantities allow us to evaluate policies and compare the actions available at each state and time.

Their recursive relationships, as stated in Theorem 5, Theorem 7, and Theorem 8, then provide the basis for dynamic programming and the application of *Bellman's principle of optimality*: whatever the initial state and first action, the remaining decisions must form an optimal policy for the state reached and the decision stages that remain. Accordingly, an optimal action maximizes the expected immediate reward plus the discounted optimal value of the next state. For a finite horizon, this principle leads to a backward induction procedure that constructs an optimal policy by solving the decision problem at each time in reverse order.

**Definition 5** (Return of a policy-dependent MDP at time  $t$ ). Let  $0 \leq \gamma \leq 1$  denote the discount factor. The *return at time  $t$  of a policy-dependent MDP* is defined as the random variable

$$g_t := \sum_{k=t+1}^T \gamma^{k-t-1} r_k \text{ for } t = 0, 1, \dots, T-1 \text{ and } g_T := 0 \tag{127}$$

•

$g_t$  thus represents the *discounted cumulative future reward* through the fixed horizon  $T$ . For  $0 \leq t \leq T-1$ ,  $\gamma = 0$  is referred to as *myopic* and entails  $g_t = r_{t+1}$ , whereas  $\gamma = 1$  gives the undiscounted finite-horizon return. Values  $\gamma < 1$  can represent a preference for earlier rewards or the downweighting of rewards whose future predictions are more uncertain.

The same return is defined for any history-dependent stochastic policy by the displayed finite sum. For any such policy  $\pi$ , the expression

$$\mathbb{E}^\pi(g_t | s_{0:t}, a_{0:t-1}) \quad (128)$$

denotes its continuation expectation after the specified admissible history, using the convention in Definition 3. At  $t = T$ , this expectation equals zero.

A basic but important property of the return is its recursive structure.

**Theorem 5** (Recursive property of the policy-dependent MDP return). *For  $t = 0, 1, \dots, T - 1$ ,*

$$g_t = r_{t+1} + \gamma g_{t+1}. \quad (129)$$

■

*Proof.* We have

$$\begin{aligned} g_t &= \sum_{k=t+1}^T \gamma^{k-t-1} r_k \\ &= r_{t+1} + \sum_{k=t+2}^T \gamma^{k-t-1} r_k \\ &= r_{t+1} + \gamma \sum_{k=t+2}^T \gamma^{k-(t+1)-1} r_k \\ &= r_{t+1} + \gamma g_{t+1}. \end{aligned} \quad (130)$$

At  $t = T - 1$ , the remaining sum is empty and equals zero, consistently with  $g_T = 0$ .

□

**Definition 6** (Value function and action-value function of a policy-dependent MDP at time  $t$ ). For every  $t \in J$ , the *value function at time  $t$  of a policy-dependent MDP* is defined as

$$v_t^\pi : S \rightarrow \mathbb{R}, s \mapsto v_t^\pi(s) := \mathbb{E}^\pi(g_t | s_t = s). \quad (131)$$

Here, the expectation is evaluated by starting in state  $s_t$  at time  $t$  and following  $\pi$  from that time onward, using the prescribed MDP transition model. This specifies  $v_t^\pi(s_t)$  for every  $s_t \in S$ , including states with  $p^\pi(s_t) = 0$ . At  $t = T$ , the return is zero, so  $v_T^\pi(s_T) = 0$  for every  $s_T \in S$ .

For every  $t = 0, 1, \dots, T - 1$ , the *action-value function at time  $t$  of a policy-dependent MDP* is defined as

$$q_t^\pi : S \times A \rightarrow \mathbb{R}, (s, a) \mapsto q_t^\pi(s, a) := \mathbb{E}^\pi(g_t | s_t = s, a_t = a). \quad (132)$$

Here, the expectation is evaluated by starting in state  $s_t$  at time  $t$ , taking action  $a_t$ , and following  $\pi$  from time  $t + 1$  onward, using the prescribed MDP transition model. This specifies  $q_t^\pi(s_t, a_t)$  for every  $(s_t, a_t) \in S \times A$ , including combinations with  $p^\pi(s_t, a_t) = 0$ .

These constructions specify versions of the conditional expectations in the displayed definitions. They agree with the ordinary conditional expectations whenever the conditioning events have positive probability.

•

**Theorem 6** (Conditional expected return under a Markovian policy). *Let  $\pi$  be the Markovian stochastic policy of a policy-dependent MDP. Then, for all  $0 \leq t \leq T - 1$  and all state-action-state combinations  $(s_t, a_t, s_{t+1}) \in S \times A \times S$  with  $p^\pi(s_t, a_t, s_{t+1}) > 0$ ,*

$$\mathbb{E}^\pi(g_{t+1} | s_t, a_t, s_{t+1}) = \mathbb{E}^\pi(g_{t+1} | s_{t+1}). \quad (133)$$

■

*Proof.*

For  $0 \leq t \leq T - 2$ ,

$$\begin{aligned} \mathbb{E}^\pi(g_{t+1} | s_t, a_t, s_{t+1}) &= \mathbb{E}^\pi \left( \sum_{k=(t+1)+1}^T \gamma^{k-(t+1)-1} r_k | s_t, a_t, s_{t+1} \right) \\ &= \mathbb{E}^\pi \left( \sum_{k=t+2}^T \gamma^{k-t-2} r_k | s_t, a_t, s_{t+1} \right) \\ &= \sum_{k=t+2}^T \gamma^{k-t-2} \mathbb{E}^\pi(r_k | s_t, a_t, s_{t+1}) \\ &= \sum_{k=t+2}^T \gamma^{k-t-2} \sum_{r_k} r_k p^\pi(r_k | s_t, a_t, s_{t+1}) \\ &\stackrel{(1)}{=} \sum_{k=t+2}^T \gamma^{k-t-2} \sum_{r_k} r_k p^\pi(r_k | s_{t+1}) \\ &= \sum_{k=t+2}^T \gamma^{k-t-2} \mathbb{E}^\pi(r_k | s_{t+1}) \\ &= \mathbb{E}^\pi \left( \sum_{k=t+2}^T \gamma^{k-t-2} r_k | s_{t+1} \right) \\ &= \mathbb{E}^\pi(g_{t+1} | s_{t+1}). \end{aligned}$$

Here, (1) applies Theorem 4. For  $t = T - 1$ , both sides equal zero because  $g_T = 0$ .

□

**Theorem 7** (Action-value function at time  $t$  and value function at time  $t + 1$ ). *Let  $v_t^\pi$  and  $q_t^\pi$  denote the value and action-value functions of a policy-dependent MDP. Then, for  $t = 0, 1, \dots, T - 1$ ,*

$$q_t^\pi(s_t, a_t) = \rho(s_t, a_t) + \gamma \sum_{s_{t+1}} p(s_{t+1}|s_t, a_t) v_{t+1}^\pi(s_{t+1}) \text{ for all } s_t \in S, a_t \in A. \quad (134)$$

■

*Proof.*

We have

$$\begin{aligned} q_t^\pi(s_t, a_t) &= \mathbb{E}^\pi(g_t | s_t, a_t) \\ &\stackrel{(1)}{=} \mathbb{E}^\pi(r_{t+1} + \gamma g_{t+1} | s_t, a_t) \\ &= \mathbb{E}^\pi(r_{t+1} | s_t, a_t) + \gamma \mathbb{E}^\pi(g_{t+1} | s_t, a_t) \\ &= \rho(s_t, a_t) + \gamma \sum_{g_{t+1}} g_{t+1} p^\pi(g_{t+1} | s_t, a_t) \\ &= \rho(s_t, a_t) + \gamma \sum_{g_{t+1}} g_{t+1} \sum_{s_{t+1}} p^\pi(g_{t+1}, s_{t+1} | s_t, a_t) \\ &= \rho(s_t, a_t) + \gamma \sum_{g_{t+1}} g_{t+1} \sum_{s_{t+1}} p^\pi(s_{t+1} | s_t, a_t) p^\pi(g_{t+1} | s_t, a_t, s_{t+1}) \\ &= \rho(s_t, a_t) + \gamma \sum_{g_{t+1}} g_{t+1} \sum_{s_{t+1}} p(s_{t+1} | s_t, a_t) p^\pi(g_{t+1} | s_t, a_t, s_{t+1}) \\ &= \rho(s_t, a_t) + \gamma \sum_{s_{t+1}} p(s_{t+1} | s_t, a_t) \sum_{g_{t+1}} g_{t+1} p^\pi(g_{t+1} | s_t, a_t, s_{t+1}) \\ &= \rho(s_t, a_t) + \gamma \sum_{s_{t+1}} p(s_{t+1} | s_t, a_t) \mathbb{E}^\pi(g_{t+1} | s_t, a_t, s_{t+1}) \\ &\stackrel{(2)}{=} \rho(s_t, a_t) + \gamma \sum_{s_{t+1}} p(s_{t+1} | s_t, a_t) \mathbb{E}^\pi(g_{t+1} | s_{t+1}) \\ &= \rho(s_t, a_t) + \gamma \sum_{s_{t+1}} p(s_{t+1} | s_t, a_t) v_{t+1}^\pi(s_{t+1}). \end{aligned}$$

Here, (1) follows from Theorem 5. Conditioning on  $a_t$  fixes the first action, so the policy-dependent next-state probability equals the state- and action-conditional transition probability of the MDP. (2) follows from Theorem 6.

□

**Theorem 8** (Value function at time  $t$  and action-value function at time  $t$ ). *Let  $v_t^\pi$  and  $q_t^\pi$  denote the value and action-value functions of a policy-dependent MDP. Then, for  $t = 0, 1, \dots, T-1$ ,*

$$v_t^\pi(s_t) = \sum_{a_t} \pi_t(s_t, a_t) q_t^\pi(s_t, a_t) \text{ for all } s_t \in S. \quad (135)$$

■

*Proof.*

We have

$$v_t^\pi(s_t) = \mathbb{E}^\pi(g_t | s_t)$$

$$\begin{aligned}
&= \sum_{g_t} g_t p^\pi(g_t | s_t) \\
&= \sum_{g_t} g_t \sum_{a_t} p^\pi(g_t, a_t | s_t) \\
&= \sum_{a_t} \sum_{g_t} g_t p^\pi(g_t | s_t, a_t) p^\pi(a_t | s_t) \\
&= \sum_{a_t} p^\pi(a_t | s_t) \sum_{g_t} g_t p^\pi(g_t | s_t, a_t) \\
&= \sum_{a_t} \pi_t(s_t, a_t) \mathbb{E}^\pi(g_t | s_t, a_t) \\
&= \sum_{a_t} \pi_t(s_t, a_t) q_t^\pi(s_t, a_t).
\end{aligned}$$

□

## Optimal value functions and policies

We now turn to the central task of these notes: establishing the existence of an optimal policy and showing how to construct it. Building on the value functions and recursive relationships developed above, we first define optimal value functions and optimal policies. The main result then shows that backward induction yields a deterministic Markovian policy that attains the optimal value at every state and time (Theorem 9). Finally, we introduce the optimal action-value function and relate it to the optimal value function.

Historically, the backward-induction construction in Theorem 9 traces to Bellman's finite-stage stochastic recursion (Bellman (1954, Section 4)), with its finite-state, finite-action form explicitly given in Bellman (1957, equation (1.1) and Section 2). Earlier contributions to sequential decision theory include Arrow et al. (1949), whose work provided part of the foundation for this development.

**Definition 7** (Optimal value function and optimal policy). Let  $\Pi$  denote the set of all Markovian stochastic policies of a finite-horizon MDP. The *optimal value function at time  $t$*  is defined as

$$v_t^* : S \rightarrow \mathbb{R}, \quad v_t^*(s_t) := \sup_{\pi \in \Pi} v_t^\pi(s_t) \text{ for all } s_t \in S \text{ and } t \in J. \quad (136)$$

A policy  $\pi^* \in \Pi$  is called *optimal* if

$$v_t^{\pi^*}(s_t) = v_t^*(s_t) \text{ for all } s_t \in S \text{ and } t \in J. \quad (137)$$

Thus, an optimal policy attains the optimal value simultaneously at every time and state.

•

The definition of  $v_t^*$  does not presuppose the existence of an optimal policy. Because the reward set and the horizon are finite, and  $0 \leq \gamma \leq 1$ , the possible returns are bounded by a finite constant independent

of the policy and the expected return is therefore bounded as well. Moreover,  $\Pi$  is nonempty because the action set is nonempty. Consequently, the supremum defining  $v_t^*(s_t)$  exists as a finite real number for all  $s_t \in S$  and  $t \in J$ .

A supremum, however, is an upper bound and need not be attained by an element of the set over which it is taken. Merely defining  $v_t^*(s_t)$  therefore does not establish that there is a policy whose expected return equals this number. In particular, the definition alone does not justify replacing the supremum by a maximum.

The following theorem establishes constructively that there is indeed a policy  $\pi^* \in \Pi$  that attains the upper bound  $\sup_{\pi \in \Pi} v_t^\pi(s)$ , i.e., that

$$v_t^{\pi^*}(s_t) = \max_{\pi \in \Pi} v_t^\pi(s_t) \text{ for all } s_t \in S \text{ and } t \in J \quad (138)$$

To this end, the theorem defines auxiliary values  $\tilde{v}_t$  by backward recursion and selects a maximizing action at each time and state. These choices determine one deterministic Markovian policy  $\pi^*$ . The proof then shows that this policy attains the computed values and that no Markovian stochastic policy has larger values. The substantive conclusion is therefore the existence and explicit construction of a single deterministic Markovian policy that attains the optimal value at every time and state. The theorem additionally establishes that history-dependent stochastic policies cannot improve upon the constructed policy after any admissible history.

**Theorem 9** (Existence and construction of an optimal policy). *Let  $\Pi$  denote the set of all Markovian stochastic policies of a finite-horizon MDP. Then there exists a deterministic Markovian policy  $\pi^* \in \Pi$  such that*

$$v_t^{\pi^*}(s_t) = \max_{\pi \in \Pi} v_t^\pi(s_t) \text{ for all } s_t \in S \text{ and } t \in J. \quad (139)$$

*In particular,  $\pi^*$  can be constructed by setting*

$$\tilde{v}_T(s_T) := 0 \text{ for all } s_T \in S, \quad (140)$$

*and, for  $t = T - 1, T - 2, \dots, 0$ , defining*

$$\tilde{v}_t(s_t) := \max_{a_t \in A} \left( \rho(s_t, a_t) + \gamma \sum_{s_{t+1}} p(s_{t+1} | s_t, a_t) \tilde{v}_{t+1}(s_{t+1}) \right) \text{ for all } s_t \in S \quad (141)$$

*and setting*

$$d_t^*(s_t) \in \arg \max_{a_t \in A} \left( \rho(s_t, a_t) + \gamma \sum_{s_{t+1}} p(s_{t+1} | s_t, a_t) \tilde{v}_{t+1}(s_{t+1}) \right) \text{ for all } s_t \in S. \quad (142)$$

Then the policy  $\pi^* := (\pi_t^*)_{t=0}^{T-1}$  defined by

$$\pi_t^*(s_t, a_t) := \begin{cases} 1 & a_t = d_t^*(s_t), \\ 0 & a_t \neq d_t^*(s_t), \end{cases} \text{ for all } t = 0, 1, \dots, T-1, s_t \in S, a_t \in A. \quad (143)$$

satisfies

$$v_t^{\pi^*}(s_t) = \tilde{v}_t(s_t) \text{ for all } s_t \in S \text{ and } t \in J. \quad (144)$$

Moreover, the constructed policy dominates every history-dependent stochastic policy after every admissible history. Specifically, for any stochastic policy  $\pi$ , including a history-dependent one,

$$\mathbb{E}^\pi(g_t | s_{0:t}, a_{0:t-1}) \leq v_t^{\pi^*}(s_t) \text{ for all } t \in J, (s_{0:t}, a_{0:t-1}) \in H_t. \quad (145)$$

■

*Proof.* We first note that because  $A$  is finite and nonempty, the maximum in equation (141) is attained and a maximizing action can be selected in equation (142). We next prove the theorem by backward induction. In doing so, we establish, at each time  $t$ ,

$$v_t^{\pi^*}(s_t) = \tilde{v}_t(s_t) \text{ for all } s_t \in S, \quad (A)$$

i.e., the expected return when starting from any state at time  $t$  and following the constructed policy  $\pi^*$  equals the value computed by the backward recursion.

Furthermore, we establish

$$v_t^\pi(s_t) \leq \tilde{v}_t(s_t) \text{ for all } \pi \in \Pi, s_t \in S, \quad (B)$$

i.e., the expected return when starting from any state at time  $t$  and following any Markovian stochastic policy is bounded above by the value computed by the backward recursion. Once established, (A) and (B), together with  $\pi^* \in \Pi$ , imply

$$\begin{aligned} v_t^{\pi^*}(s_t) = \tilde{v}_t(s_t) &\geq v_t^\pi(s_t) \text{ for all } \pi \in \Pi, s_t \in S, t \in J \\ &\Rightarrow v_t^{\pi^*}(s_t) = \max_{\pi \in \Pi} v_t^\pi(s_t) = v_t^*(s_t) \text{ for all } s_t \in S \text{ and } t \in J. \end{aligned} \quad (146)$$

Note that equation (146) explicitly establishes attainment, which justifies replacing the supremum in the definition of  $v_t^*$  by a maximum.

Finally, for any stochastic policy  $\pi$  (including history-dependent policies), we establish

$$\mathbb{E}^\pi(g_t | s_{0:t}, a_{0:t-1}) \leq \tilde{v}_t(s_t) \text{ for all } (s_{0:t}, a_{0:t-1}) \in H_t, \quad (C)$$

i.e., the expected return when continuing from any admissible history under any stochastic policy is bounded above by the value computed by the backward recursion at the history's final state. The conditional expectations in (C) are understood as continuation expectations after the specified admissible histories. Terms with zero probability weight may be omitted from the sums below. Once established, (A) and (C) imply, for any stochastic policy  $\pi$ , including a history-dependent one,

$$\mathbb{E}^\pi(g_t | s_{0:t}, a_{0:t-1}) \leq v_t^{\pi^*}(s_t) \text{ for all } t \in J, (s_{0:t}, a_{0:t-1}) \in H_t. \quad (147)$$

It therefore remains to establish (A), (B), and (C) by backward induction.

*Base case.* Let  $t = T$ . For the constructed policy  $\pi^*$ , we have

$$v_T^{\pi^*}(s_T) = \mathbb{E}^{\pi^*}(g_T | s_T) = \mathbb{E}^{\pi^*}(0 | s_T) = 0 \text{ for all } s_T \in S. \quad (148)$$

Similarly, for any Markovian stochastic policy  $\pi \in \Pi$ ,

$$v_T^\pi(s_T) = \mathbb{E}^\pi(g_T | s_T) = \mathbb{E}^\pi(0 | s_T) = 0 \text{ for all } \pi \in \Pi, s_T \in S. \quad (149)$$

By the terminal initialization of the backward recursion,

$$\tilde{v}_T(s_T) := 0 \text{ for all } s_T \in S. \quad (150)$$

Consequently,

$$v_T^{\pi^*}(s_T) = \tilde{v}_T(s_T) \text{ for all } s_T \in S, \quad (151)$$

and

$$v_T^\pi(s_T) \leq \tilde{v}_T(s_T) \text{ for all } \pi \in \Pi, s_T \in S. \quad (152)$$

Finally, for any stochastic policy  $\pi$ , including a history-dependent one,

$$\mathbb{E}^\pi(g_T | s_{0:T}, a_{0:T-1}) = \mathbb{E}^\pi(0 | s_{0:T}, a_{0:T-1}) = 0 = \tilde{v}_T(s_T) \text{ for all } (s_{0:T}, a_{0:T-1}) \in H_T. \quad (153)$$

Thus, (A), (B), and (C) hold at time  $T$ .

*Induction step.* Let  $t \in \{0, 1, \dots, T-1\}$ , and assume that (A), (B), and (C) hold at time  $t+1$ .

For (A), the induction hypothesis states that

$$v_{t+1}^{\pi^*}(s_{t+1}) = \tilde{v}_{t+1}(s_{t+1}) \text{ for all } s_{t+1} \in S. \quad (154)$$

Then, for all  $s_t \in S$ , we have

$$\begin{aligned} v_t^{\pi^*}(s_t) &\stackrel{(1)}{=} \sum_{a_t} \pi_t^*(s_t, a_t) q_t^{\pi^*}(s_t, a_t) \\ &\stackrel{(2)}{=} q_t^{\pi^*}(s_t, d_t^*(s_t)) \\ &\stackrel{(3)}{=} \rho(s_t, d_t^*(s_t)) + \gamma \sum_{s_{t+1}} p(s_{t+1} | s_t, d_t^*(s_t)) v_{t+1}^{\pi^*}(s_{t+1}) \\ &\stackrel{(4)}{=} \rho(s_t, d_t^*(s_t)) + \gamma \sum_{s_{t+1}} p(s_{t+1} | s_t, d_t^*(s_t)) \tilde{v}_{t+1}(s_{t+1}) \\ &\stackrel{(5)}{=} \tilde{v}_t(s_t). \end{aligned}$$

Here, (1) follows from Theorem 8, (2) uses the deterministic policy defined by the decision function  $d_t^*$  in equation (143), (3) follows from Theorem 7, (4) applies induction hypothesis (A), and (5) uses the maximizing choice of  $d_t^*$  in equation (142) and the definition of  $\tilde{v}_t$  in equation (141). This proves (A) at time  $t$ .

For (B), the induction hypothesis states that

$$v_{t+1}^\pi(s_{t+1}) \leq \tilde{v}_{t+1}(s_{t+1}) \text{ for all } \pi \in \Pi, s_{t+1} \in S. \quad (155)$$

Then, for all  $\pi \in \Pi$  and  $s_t \in S$ , we have

$$\begin{aligned} v_t^\pi(s_t) &\stackrel{(1)}{=} \sum_{a_t} \pi_t(s_t, a_t) q_t^\pi(s_t, a_t) \\ &\stackrel{(2)}{=} \sum_{a_t} \pi_t(s_t, a_t) \left( \rho(s_t, a_t) + \gamma \sum_{s_{t+1}} p(s_{t+1} | s_t, a_t) v_{t+1}^\pi(s_{t+1}) \right) \\ &\stackrel{(3)}{\leq} \sum_{a_t} \pi_t(s_t, a_t) \left( \rho(s_t, a_t) + \gamma \sum_{s_{t+1}} p(s_{t+1} | s_t, a_t) \tilde{v}_{t+1}(s_{t+1}) \right) \\ &\stackrel{(4)}{\leq} \sum_{a_t} \pi_t(s_t, a_t) \max_{a \in A} \left( \rho(s_t, a) + \gamma \sum_{s_{t+1}} p(s_{t+1} | s_t, a) \tilde{v}_{t+1}(s_{t+1}) \right) \end{aligned}$$

$$\begin{aligned}
&\stackrel{(5)}{=} \sum_{a_t} \pi_t(s_t, a_t) \tilde{v}_t(s_t) \\
&= \tilde{v}_t(s_t) \sum_{a_t} \pi_t(s_t, a_t) \\
&= \tilde{v}_t(s_t) \sum_{a_t} p^\pi(a_t | s_t) \\
&= \tilde{v}_t(s_t).
\end{aligned}$$

Here, (1) follows from Theorem 8, (2) follows from Theorem 7, (3) applies induction hypothesis (B), using the nonnegativity of  $\gamma$  and the probability weights, (4) bounds each action-specific expression by its maximum over all actions, and (5) uses the definition of  $\tilde{v}_t$  in equation (141). The variable  $a$  in (4) is distinct from the outer summation variable  $a_t$ . The last equality holds because the conditional action probabilities sum to one. This proves (B) at time  $t$ .

For (C), the induction hypothesis states that, for any stochastic policy  $\pi$ , including a history-dependent one,

$$\mathbb{E}^\pi(g_{t+1} | s_{0:t+1}, a_{0:t}) \leq \tilde{v}_{t+1}(s_{t+1}) \text{ for all } (s_{0:t+1}, a_{0:t}) \in H_{t+1}. \quad (156)$$

Fix any such policy  $\pi$ . Then, for all  $(s_{0:t}, a_{0:t-1}) \in H_t$ , we have

$$\begin{aligned}
\mathbb{E}^\pi(g_t | s_{0:t}, a_{0:t-1}) &\stackrel{(1)}{=} \mathbb{E}^\pi(r_{t+1} + \gamma g_{t+1} | s_{0:t}, a_{0:t-1}) \\
&= \mathbb{E}^\pi(r_{t+1} | s_{0:t}, a_{0:t-1}) + \gamma \mathbb{E}^\pi(g_{t+1} | s_{0:t}, a_{0:t-1}) \\
&= \sum_{a_t} p^\pi(a_t | s_{0:t}, a_{0:t-1}) (\mathbb{E}^\pi(r_{t+1} | s_{0:t}, a_{0:t}) + \gamma \mathbb{E}^\pi(g_{t+1} | s_{0:t}, a_{0:t})) \\
&= \sum_{a_t} p^\pi(a_t | s_{0:t}, a_{0:t-1}) \left( \sum_{r_{t+1}} r_{t+1} p^\pi(r_{t+1} | s_{0:t}, a_{0:t}) + \gamma \mathbb{E}^\pi(g_{t+1} | s_{0:t}, a_{0:t}) \right) \\
&= \sum_{a_t} p^\pi(a_t | s_{0:t}, a_{0:t-1}) \left( \sum_{r_{t+1}} \sum_{s_{t+1}} r_{t+1} p^\pi(r_{t+1}, s_{t+1} | s_{0:t}, a_{0:t}) + \gamma \mathbb{E}^\pi(g_{t+1} | s_{0:t}, a_{0:t}) \right) \\
&\stackrel{(2)}{=} \sum_{a_t} p^\pi(a_t | s_{0:t}, a_{0:t-1}) \left( \sum_{r_{t+1}} \sum_{s_{t+1}} r_{t+1} p(r_{t+1}, s_{t+1} | s_{0:t}, a_{0:t}) + \gamma \mathbb{E}^\pi(g_{t+1} | s_{0:t}, a_{0:t}) \right) \\
&\stackrel{(3)}{=} \sum_{a_t} p^\pi(a_t | s_{0:t}, a_{0:t-1}) \left( \sum_{r_{t+1}} \sum_{s_{t+1}} r_{t+1} p(r_{t+1}, s_{t+1} | s_t, a_t) + \gamma \mathbb{E}^\pi(g_{t+1} | s_{0:t}, a_{0:t}) \right) \\
&= \sum_{a_t} p^\pi(a_t | s_{0:t}, a_{0:t-1}) (\mathbb{E}(r_{t+1} | s_t, a_t) + \gamma \mathbb{E}^\pi(g_{t+1} | s_{0:t}, a_{0:t})) \\
&\stackrel{(4)}{=} \sum_{a_t} p^\pi(a_t | s_{0:t}, a_{0:t-1}) (\rho(s_t, a_t) + \gamma \mathbb{E}^\pi(g_{t+1} | s_{0:t}, a_{0:t})) \\
&= \sum_{a_t} p^\pi(a_t | s_{0:t}, a_{0:t-1}) \left( \rho(s_t, a_t) + \gamma \sum_{g_{t+1}} g_{t+1} p^\pi(g_{t+1} | s_{0:t}, a_{0:t}) \right) \\
&= \sum_{a_t} p^\pi(a_t | s_{0:t}, a_{0:t-1}) \left( \rho(s_t, a_t) + \gamma \sum_{s_{t+1}} \sum_{g_{t+1}} g_{t+1} p^\pi(g_{t+1}, s_{t+1} | s_{0:t}, a_{0:t}) \right) \\
&= \sum_{a_t} p^\pi(a_t | s_{0:t}, a_{0:t-1}) \left( \rho(s_t, a_t) + \gamma \sum_{s_{t+1}} \sum_{g_{t+1}} g_{t+1} p^\pi(g_{t+1} | s_{0:t+1}, a_{0:t}) p^\pi(s_{t+1} | s_{0:t}, a_{0:t}) \right) \\
&= \sum_{a_t} p^\pi(a_t | s_{0:t}, a_{0:t-1}) \left( \rho(s_t, a_t) + \gamma \sum_{s_{t+1}} p^\pi(s_{t+1} | s_{0:t}, a_{0:t}) \mathbb{E}^\pi(g_{t+1} | s_{0:t+1}, a_{0:t}) \right) \\
&\stackrel{(5)}{=} \sum_{a_t} p^\pi(a_t | s_{0:t}, a_{0:t-1}) \left( \rho(s_t, a_t) + \gamma \sum_{s_{t+1}} p(s_{t+1} | s_t, a_t) \mathbb{E}^\pi(g_{t+1} | s_{0:t+1}, a_{0:t}) \right) \\
&\stackrel{(6)}{\leq} \sum_{a_t} p^\pi(a_t | s_{0:t}, a_{0:t-1}) \left( \rho(s_t, a_t) + \gamma \sum_{s_{t+1}} p(s_{t+1} | s_t, a_t) \tilde{v}_{t+1}(s_{t+1}) \right)
\end{aligned}$$

$$\begin{aligned}
&\stackrel{(7)}{\leq} \sum_{a_t} p^\pi(a_t|s_{0:t}, a_{0:t-1}) \max_{a \in A} \left( \rho(s_t, a) + \gamma \sum_{s_{t+1}} p(s_{t+1}|s_t, a_t = a) \tilde{v}_{t+1}(s_{t+1}) \right) \\
&\stackrel{(8)}{=} \sum_{a_t} p^\pi(a_t|s_{0:t}, a_{0:t-1}) \tilde{v}_t(s_t) \\
&= \tilde{v}_t(s_t) \sum_{a_t} p^\pi(a_t|s_{0:t}, a_{0:t-1}) \\
&= \tilde{v}_t(s_t).
\end{aligned}$$

Here, (1) follows from Theorem 5, whose return identity also holds for history-dependent policies and (2) uses the construction of the policy-induced process from the prescribed MDP transition model: once the history and current action are fixed, the next reward and state are generated by that model. (3) applies the conditional independence property of the state-reward probabilities in equation (90) and (4) uses the definition of  $\rho$  in equation (95). (5) uses the prescribed MDP transition model and the conditional independence property of the next-state probabilities in equation (93). The continuation expectation itself remains conditional on the full extended history. (6) then applies the induction hypothesis (C), using the nonnegativity of  $\gamma$  and the probability weights. Every successor history contributing a positive probability weight is admissible, so the induction hypothesis applies to each such term. (7) bounds each action-specific expression by its maximum over all actions, and (8) uses the definition of  $\tilde{v}_t$  in equation (141). The last equality holds because the conditional action probabilities sum to one. This proves (C) at time  $t$ . □

**Definition 8** (Optimal action-value function). Let  $\Pi$  denote the set of all Markovian stochastic policies of a finite-horizon MDP. For  $t = 0, 1, \dots, T-1$ , the *optimal action-value function at time  $t$*  is defined as

$$q_t^* : S \times A \rightarrow \mathbb{R}, \quad q_t^*(s_t, a_t) := \sup_{\pi \in \Pi} q_t^\pi(s_t, a_t) \text{ for all } (s_t, a_t) \in S \times A. \quad (157)$$

•

The optimal action-value function is related to the optimal value function by the identities stated in the following theorem.

**Theorem 10** (Optimal action-value function and optimal value function). *Let  $v_t^*$  and  $q_t^*$  denote the optimal value function and the optimal action-value function of a finite-horizon MDP. Then, for all  $s_t \in S$ ,  $a_t \in A$ , and  $t = 0, 1, \dots, T-1$ ,*

$$q_t^*(s_t, a_t) = \rho(s_t, a_t) + \gamma \sum_{s_{t+1}} p(s_{t+1}|s_t, a_t) v_{t+1}^*(s_{t+1}) \quad (158)$$

and

$$v_t^*(s_t) = \max_{a_t \in A} q_t^*(s_t, a_t). \quad (159)$$

■

*Proof.* By definition and with Theorem 7, we have

$$\begin{aligned}
q_t^*(s_t, a_t) &= \sup_{\pi \in \Pi} q_t^\pi(s_t, a_t) \\
&= \sup_{\pi \in \Pi} \left( \rho(s_t, a_t) + \gamma \sum_{s_{t+1}} p(s_{t+1}|s_t, a_t) v_{t+1}^\pi(s_{t+1}) \right) \\
&= \rho(s_t, a_t) + \gamma \sum_{s_{t+1}} p(s_{t+1}|s_t, a_t) v_{t+1}^*(s_{t+1}).
\end{aligned} \tag{160}$$

Here, the last equality follows from Theorem 9 as every policy  $\pi \in \Pi$  satisfies  $v_{t+1}^\pi(s_{t+1}) \leq v_{t+1}^*(s_{t+1})$ , and the policy  $\pi^*$  attains equality simultaneously for all successor states. Since the discount factor and transition probabilities are nonnegative, this policy also attains the supremum of the weighted sum. Further, in Theorem 9, we have seen that

$$\tilde{v}_t(s_t) = v_t^*(s_t) \text{ for all } s_t \in S \text{ and } t \in J. \tag{161}$$

Substitution in equation (141) together with equation (158) thus yields for all  $s_t \in S$  and  $t = 0, 1, \dots, T-1$

$$\begin{aligned}
v_t^*(s_t) &= \max_{a_t \in A} \left( \rho(s_t, a_t) + \gamma \sum_{s_{t+1}} p(s_{t+1}|s_t, a_t) v_{t+1}^*(s_{t+1}) \right) \\
&= \max_{a_t \in A} q_t^*(s_t, a_t).
\end{aligned} \tag{162}$$

□

## S2 Experimental statistics validation

### Purpose and validation design

The state transition model specifies how a new stimulus is generated after the participant selects one of the two tasks. Its observable implications can therefore be examined without fitting an agent model. We used the experimental PTD-RMC data to estimate the transition frequencies of the stimulus labels and identifiers, and compared them with the corresponding frequencies obtained from simulations of the full MDP. The aim was not to reproduce participants’ task-allocation behavior. It was to determine whether the state statistics generated by the model resemble those implemented in the experiment.

The comparison was restricted to PTD and RMC, which are the two tasks of the paradigm analyzed in the main text. All 49 participants were included. Their original run and block structures were retained, as were the task-specific stimulus catalogues, the participant- and block-specific PTD target sets, the fixed RMC categories, and the number of trials observed in each block. This design matching is important because pooled frequencies can otherwise differ merely because target assignments or block lengths differ between an experimental and a simulated sample.

For the simulation, actions were generated exclusively by A0. At every trial, A0 selects uniformly from the four task-response actions. No fitted parameters and no optimal-policy computations enter this validation. The selected action determines which stimulus stream is advanced, while the full MDP generates the successor state according to its transition distribution. This provides a direct simulation of the model component under examination and is much faster than generating the same states through A1.

The use of A0 is also conceptually appropriate. Conditional on the selected task, the transition law for the new label and identifier does not depend on why that task was selected. Different action policies change how often PTD or RMC is sampled in a finite block, but they do not change the corresponding conditional stimulus distribution. With sufficiently many transitions, the estimated state statistics should therefore not depend on whether actions were generated by A0, A1, or a participant. Uniform random actions provide balanced exposure to both streams without importing a behavioral model into the validation.

The observed and simulated data were processed by the same counting function. Thus, differences between the two sets of figures cannot arise from using different definitions, recodings, or normalization procedures. A fixed random seed makes the simulated comparison reproducible. The simulation remains one finite realization of the model, so exact numerical equality is neither expected nor required.

## Transition reconstruction and frequency estimates

Each data row records the state visible before an action. A selected-stream transition was reconstructed by pairing this state with the next state in the same participant, run, and block. Transitions were never formed across block boundaries or across gaps in the trial index. The terminal action of a block has no recorded successor and consequently contributes no transition. A missing stimulus in the unselected stream does not exclude an otherwise known update of the selected stream.

Stimulus labels were recovered from the experimental design rather than from participants' responses. For PTD, the three targets were read separately for each participant and block. The remaining 15 patterns received label 0. For RMC, the four rotation-only stimuli and the four mirrored-and-rotated stimuli form fixed label classes. Responses were retained when the label-frequency panels were split into No and Yes actions. They were pooled for the identifier matrices because the transition law depends on the selected task but not on the response component of its action.

The upper two panels of each figure show the estimated probability of a positive successor label conditional on the current identifier and response. For task  $j$  and response  $k$ , the displayed estimand is

$$\hat{p}(s_{t+1}^{j,1} = 1 | s_t^{j,2} = i, a_t = a^{j,k}). \quad (163)$$

The two successor labels are generated with equal probability by the model. Accordingly, the bars should fluctuate around 1/2. Separate No and Yes panels retain potentially informative finite-sample differences without interpreting the response as a cause of the next stimulus.

The remaining four panels condition on the current label  $o$  and successor label  $l$ . Their matrix entries estimate

$$\hat{p}(s_{t+1}^{j,2} = h | s_{t+1}^{j,1} = l, s_t^{j,1} = o, s_t^{j,2} = i, a_t \in A^j). \quad (164)$$

Rows represent current identifiers and columns represent successor identifiers. Each observed row is normalized separately. Direct repetitions of the selected stream's identifier are excluded by the full MDP. Within the remaining label-specific support, the successor identifier is sampled uniformly. The matrices therefore display both structural zeros and approximately equal probabilities over eligible cells.

RMC has a fixed label map, so all four matrices use the original identifiers 1 through 8. The PTD target set varies across blocks. For panels with a target successor, block-specific identifiers were therefore recoded into three target ranks followed by 15 non-target ranks before pooling. A tilde marks these rank coordinates. The same blockwise permutation was applied to the current and successor identifiers. Hatched rows represent incompatible conditioning events rather than estimated zero probabilities.

## Simulation comparison and interpretation

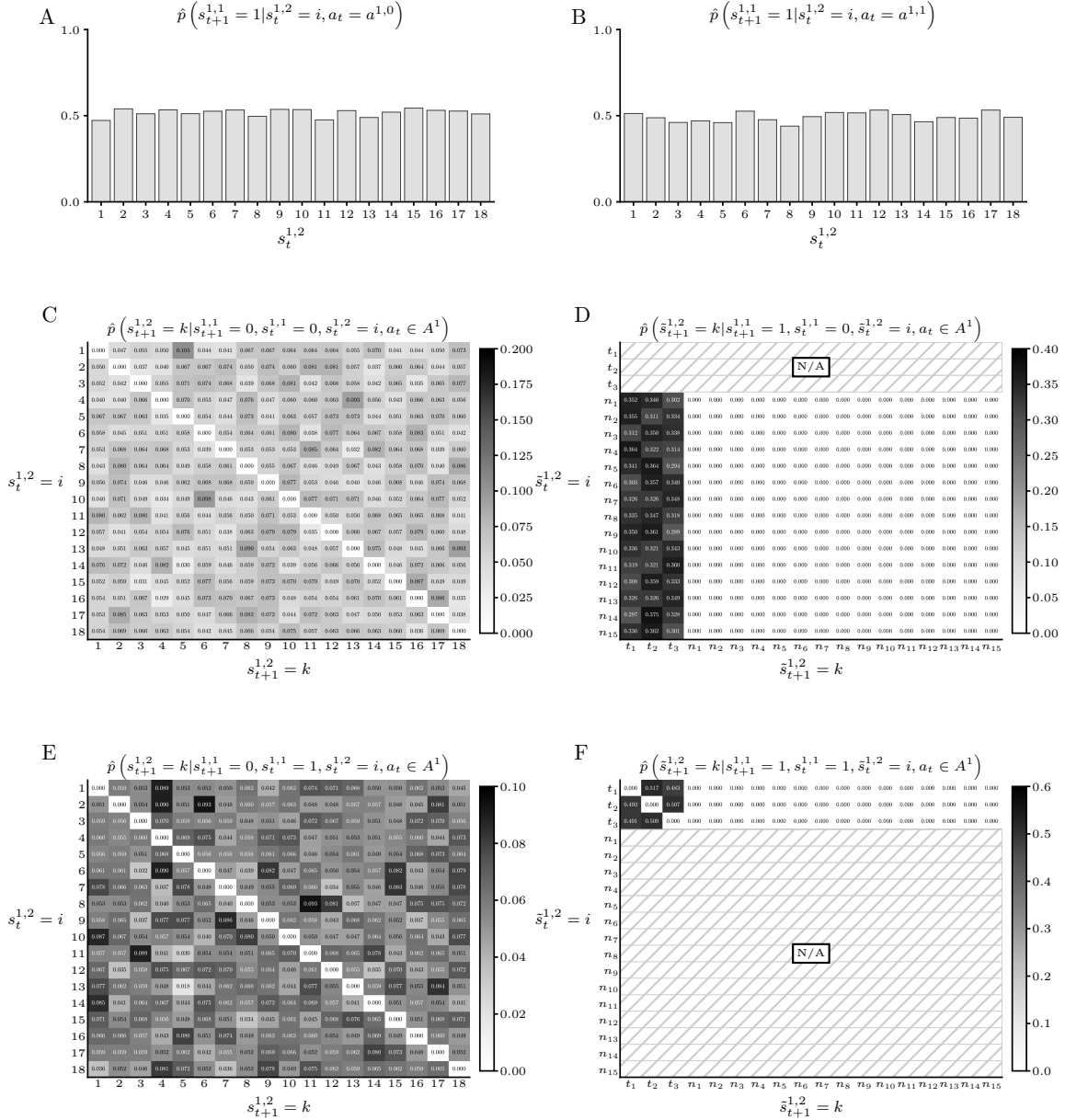
The validation simulation reproduced every PTD-RMC block in the experimental design. Each simulated block used its original stimulus catalogue, PTD target set, RMC category assignments, and observed number of trials. The initial state and every successor state were sampled from the full MDP. Only the action generator was simplified to A0. The resulting simulated cohort has the same participants and blockwise observation opportunities as the experimental cohort, apart from the random allocation of actions and the terminal transition that is unobserved in both data sources.

The comparison is descriptive and focuses on the probability patterns that are visible in the pooled data. For PTD, the experimental and simulated label frequencies are both concentrated near  $1/2$ . The identifier matrices show the same exclusion of direct repetitions and the same division between target and non-target supports. The target-successor panels also reproduce the expected probabilities over the three target ranks after the block-specific target sets have been aligned. Figure S1A and Figure S1B show the experimental and simulated PTD results.

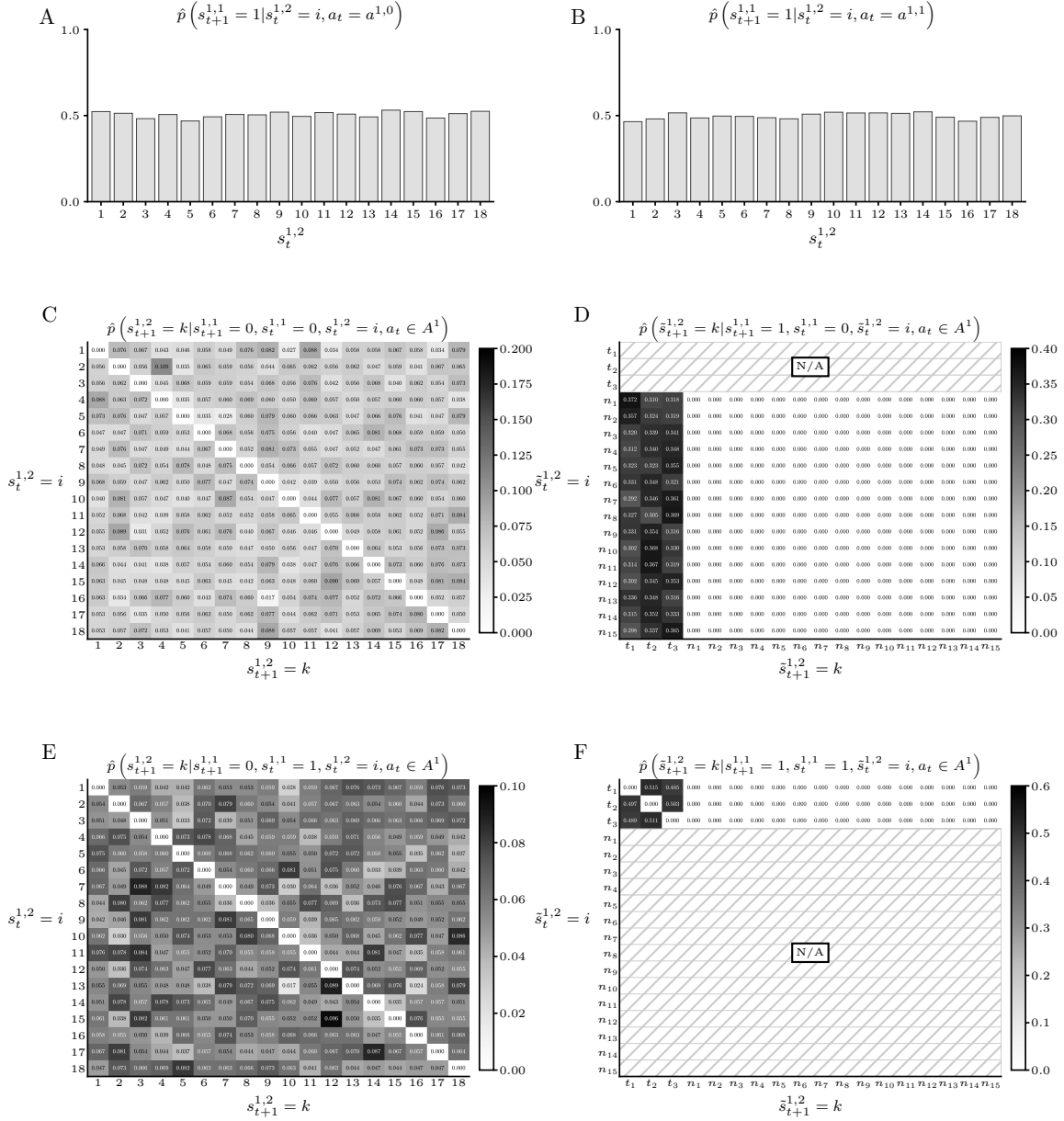
The RMC comparison yields the same conclusion. Both data sources preserve the fixed four-by-four label partition. A label change leaves four eligible successor identifiers, while retaining the same label leaves three because the current identifier is excluded. The corresponding probabilities are therefore concentrated near  $1/4$  and  $1/3$ , respectively. The remaining variation is consistent with finite counts in the individual conditioning rows. Figure S1C and Figure S1D show these results.

Agreement in these pooled summaries validates the experimentally relevant stimulus-generation statistics of the full MDP. It does not by itself test every conditional independence statement at the level of complete states. Many fully specified states are sparsely observed, and pooling can conceal small context-specific discrepancies. It also does not validate A0 as a behavioral account. A0 is used only to create action-independent exposure to the transition kernel. Participant behavior and the comparison of A0 with A1 remain separate questions addressed by the agent-model analysis.

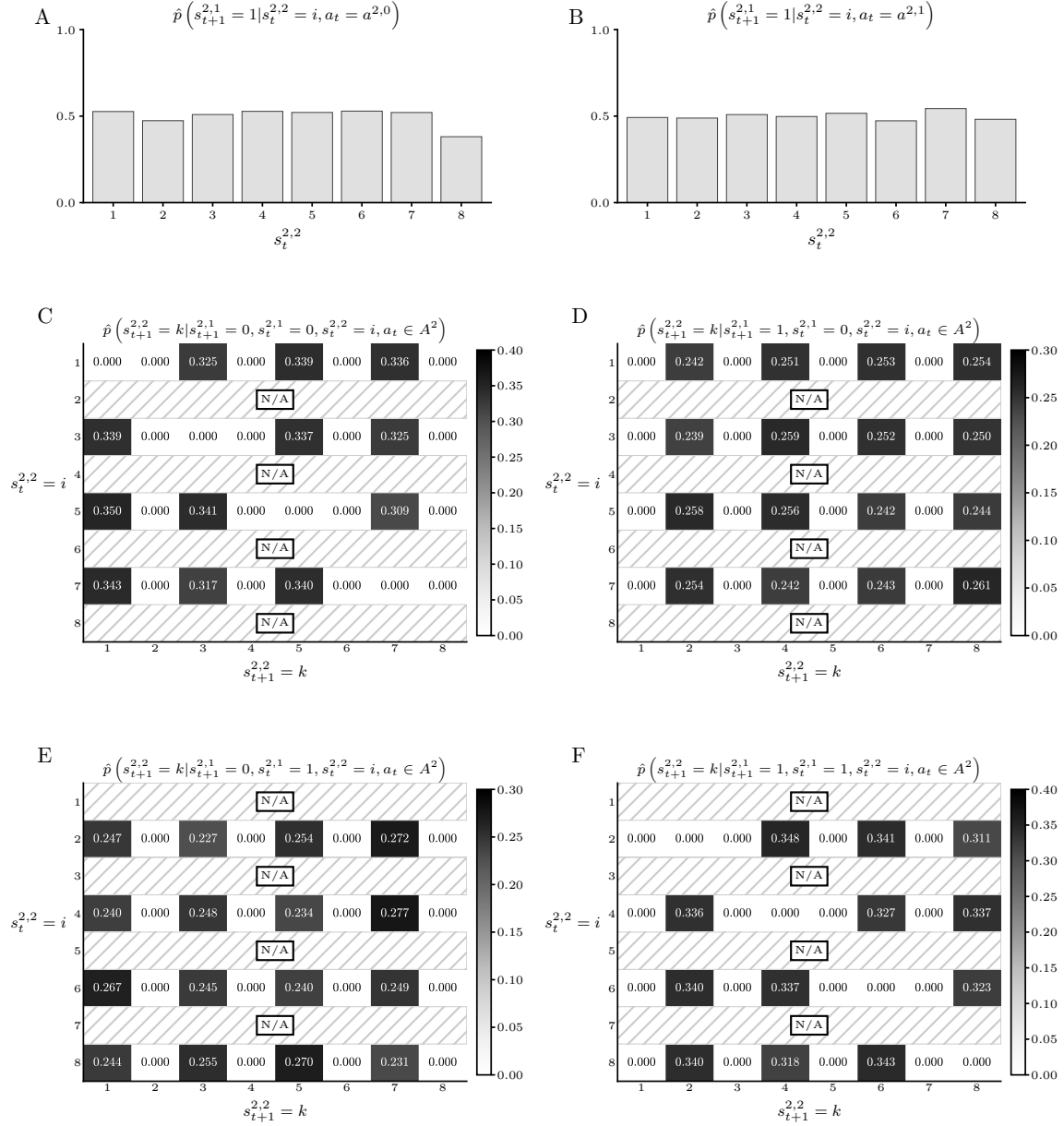
The analysis applies the same counting function to observed and simulated trials. The associated numerical observed-simulated discrepancies are exported with the validation results. For implementational details, please refer to *abm\_validation.py*.



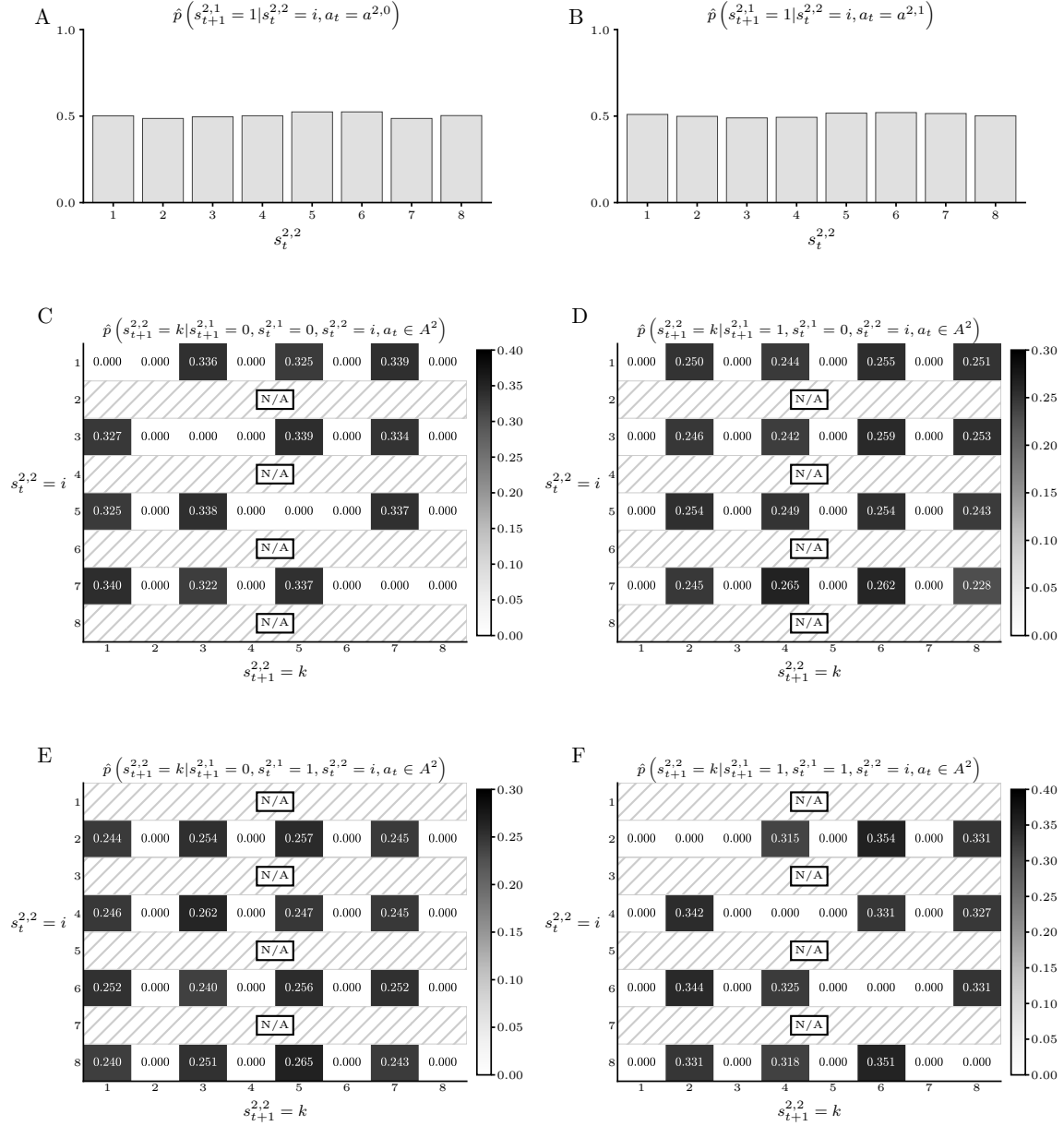
**Figure S1A.** Observed PTD transition frequencies. Panels A and B show the probability of a target successor after No and Yes responses. Panels C–F show conditional identifier frequencies for the four combinations of current and successor labels. For details, please refer to *abm\_figure\_S1.py*.



**Figure S1B.** PTD transition frequencies generated from the full MDP with A0 actions and the participant-specific experimental designs. The panel definitions match Figure S1A. For details, please refer to *abm\_figure\_S1.py*.



**Figure S1C.** Observed RMC transition frequencies. Panels A and B show the probability of a rotation-only successor after No and Yes responses. Panels C–F show conditional identifier frequencies for the four combinations of current and successor labels. For details, please refer to *abm\_figure\_S1.py*.



**Figure S1D.** RMC transition frequencies generated from the full MDP with A0 actions and the participant-specific experimental designs. The panel definitions match Figure S1C. For details, please refer to *abm\_figure\_S1.py*.

### S3 Implementational notes

#### Successor indices for backward induction

The reduced transition distribution assigns probability one to a single successor for each state-action pair. We exploit this by storing the reduced states as rows of the array `mdp_chi.S` and recording the row index of each successor in `mdp_chi.k`. Each row of `mdp_chi.k` has two entries, corresponding to the two abstract task actions. State row indices are distinct from the decision-stage index  $t$ . Row zero represents the initial state, while the remaining rows enumerate the regular states in  $T \times B$ .

Write  $s_{(i)}$  for the state in row  $i$ , let  $k(i, a)$  denote the row of its unique successor under action  $a$ , and store continuation values as  $V_t(i) := v_t(s_{(i)})$ . The expectation appearing in backward induction then satisfies

$$\sum_{s_{t+1}} p_\chi(s_{t+1} | s_t = s_{(i)}, a_t = a) v_{t+1}(s_{t+1}) = V_{t+1}(k(i, a)). \quad (165)$$

Thus, evaluating the expected continuation value requires a single array lookup for each state-action pair. There is no need to construct a dense transition matrix, sum over zero probabilities, or search the state array for the successor during the backward sweep. At an absorbing boundary,  $k(i, a) = i$  for either action. This is an exact computational representation of the probability distribution in the reduction theorem, so the general backward-induction theorem applies without modification.

For the implementation, we decompose the reduced reward as

$$\rho_\chi(s, a) = r_a(s, a) + \beta r_b(s, a) + \sigma r_s(s, a). \quad (166)$$

The parameter-independent component functions  $r_a, r_b, r_s : S_\chi \times A_\chi \rightarrow \mathbb{R}$  are defined by

$$(s, a) \mapsto r_a(s, a) := 1, \quad (167)$$

$$(s, a) \mapsto r_b(s, a) := -|s^4 - (-1)^a|, \quad (168)$$

and

$$(s, a) \mapsto r_s(s, a) := -\llbracket (s^3 = 1 \wedge a = 2) \vee (s^3 = 2 \wedge a = 1) \rrbracket. \quad (169)$$

The successor-index table and the arrays representing  $r_a$ ,  $r_b$ , and  $r_s$  depend on the reduced model's state space but not on  $\beta$  or  $\sigma$ . They can therefore be prepared once and reused across parameter evaluations. For each parameter pair, only the combined rewards and the backward sweep need to be recomputed. The resulting sweep requires  $O(n|S_\chi||A_\chi|)$  operations, with  $|A_\chi| = 2$ , rather than an additional sum over all successor states at each update.

```

"""
Compute an optimal task policy for the reduced FCDT-MDP.
"""

import numpy as np                                     # numerical arrays

def abm_dp(mdp_chi):                                   # finite-horizon optimal policy
    """
    Apply backward induction to the reduced FCDT-MDP with gamma = 1.

    Input
    mdp_chi : SimpleNamespace() object with required attributes:
        .n : positive integer horizon
        .A : ordered task-action array containing one and two
        .k : integer successor indices, one column per task action
        .r_a : parameter-independent accuracy reward array
        .r_b : parameter-independent imbalance reward array
        .r_s : parameter-independent switching reward array
        .beta : finite signed imbalance cost parameter
        .sigma : nonnegative finite switching cost parameter

    Output
    mdp_chi : input SimpleNamespace with the new attributes:
        .p : reward array for the supplied cost parameters
        .d : uint8 decision rules representing the optimal policy pi*
        .v : optimal values at stage zero for all reduced states
        .qx : action values indexed by stage, state row, and task action

    [Additional documentation]
    """
    # numerical arrays
    # -----
    n = int(mdp_chi.n)                                # number of decision stages
    n_s = mdp_chi.k.shape[0]                           # reduced state-space cardinality
    mdp_chi.p = np.full((n_s, 2), np.nan, dtype = np.float64) # p_X(s,a)
    mdp_chi.d = np.zeros((n, n_s), dtype = np.uint8)    # decision rules d_t*(s) representing pi*
    mdp_chi.v = np.full(n_s, np.nan, dtype = np.float64) # first state-value buffer

    # reward evaluation
    # -----
    mdp_chi.p[:] = (mdp_chi.r_a + mdp_chi.beta * mdp_chi.r_b # accuracy and imbalance rewards
                   + mdp_chi.sigma * mdp_chi.r_s)           # switching reward

    # terminal initialization
    # -----
    mdp_chi.v[:] = 0.0                                     # v_n(s) = 0 for every state
    v_tt = mdp_chi.v                                       # continuation values v_{t+1}
    v_t = np.full(n_s, np.nan, dtype = np.float64)        # local buffer for current state values v_t
    mdp_chi.qx = np.empty((n, n_s, 2), dtype = np.float64) # reduced action values q_{X,t}(s,a)
    q_t = np.full((n_s, 2), np.nan, dtype = np.float64)  # current action values q_{X,t}(s,a)

    # backward induction
    # -----
    for t in range(n - 1, -1, -1):                         # stages n-1 through zero
        q_t[:] = mdp_chi.p + v_tt[mdp_chi.k]               # q_{X,t}(s,a) = p_X(s,a) + v_{t+1}(s_{t+1})
        mdp_chi.qx[t] = q_t                               # retain current reduced action values
        mdp_chi.d[t] = mdp_chi.A[np.argmax(q_t, axis = 1)] # optimal decision function values d_t*(s)
        v_t[:] = np.max(q_t, axis = 1)                    # v_t*(s) = max_a q_t*(s,a)
        v_tt, v_t = v_t, v_tt                             # exchange buffers for the preceding stage

    # output
    # -----
    mdp_chi.v = v_t                                         # optimal state values v_0
    return mdp_chi                                          # optimal policy pi* represented by .d, and values

```

## Relation of the code to the optimal-policy theorem

The function `abm_dp` implements the construction in Theorem 9, “Existence and construction of an optimal policy,” of the Supplementary Information for the reduced FCDT-MDP. The theorem’s horizon is set to  $n$  and its discount factor to  $\gamma = 1$ , so the objective is the expected sum of rewards over the remaining decision stages. The function receives the reduced-model arrays prepared by `abm_mdp_chi`, together with the parameters  $\beta$  and  $\sigma$ . It computes the optimal policy for this reduced model. Establishing that the policy lifted to the original FCDT-MDP is also optimal requires the separate equivalence proof.

The rows of the numerical arrays enumerate states rather than time points. There are  $|S_\chi| = 4n + 3$  such rows, including the initial state in row zero. The two columns of the reward and action-value arrays correspond to the two actions in `mdp_chi.A`. The array `mdp_chi.d` additionally has a stage dimension, because the optimal decision function may depend on the number of actions remaining. Its shape is therefore  $n$  by  $|S_\chi|$ . There is no decision-function row for stage  $n$ , at which no further action occurs.

The estimation domain allows a finite signed imbalance parameter and a finite nonnegative switching-cost parameter. The reward calculation combines the three precomputed arrays according to

$$\rho_\chi(s, a) = r_a(s, a) + \beta r_b(s, a) + \sigma r_s(s, a). \quad (170)$$

This calculation occurs once per function call, outside the backward loop, because the immediate reward is time homogeneous. The state space, successor indices, and parameter-independent reward components can remain unchanged when evaluating another parameter pair.

The assignment of zero to every entry of `mdp_chi.v` implements the terminal condition  $\tilde{v}_n(s) = 0$  in Theorem 9. At the start of each backward iteration, `v_tt` contains  $\tilde{v}_{t+1}$  and `v_t` is the buffer into which  $\tilde{v}_t$  will be written. The Python loop visits stages  $n - 1, n - 2, \dots, 0$  in that order. Consequently, the required continuation values have already been computed when a stage is evaluated.

The expression `v_tt[mdp_chi.k]` gathers the continuation value of the unique successor of every state-action pair. Its result has the same shape as the reward array. Adding the reward therefore evaluates

$$q_t(s_t, a_t) := \rho_\chi(s_t, a_t) + \sum_{s_{t+1}} p_\chi(s_{t+1} | s_t, a_t) \tilde{v}_{t+1}(s_{t+1}) \quad (171)$$

simultaneously for all states and both actions. Although the code uses a lookup instead of an explicit probability sum, it evaluates precisely the expectation in Theorem 9. The probability of the stored successor is one, and all other successor probabilities are zero. No alternative theorem for deterministic transitions is needed.

The next line applies `np.argmax` along the action dimension. It returns the column of a maximizing action for each state. Indexing `mdp_chi.A` with these columns translates the zero-based column indices

into the actual action values 1 and 2. The resulting entries of `mdp_chi.d` are the values of the optimal decision function:

$$d_t^*(s) \in \arg \max_{a \in A_\chi} q_t(s, a). \quad (172)$$

When both action values are exactly equal, `np.argmax` returns the first column, so the code selects Task 1. This is a permissible choice among maximizers in the theorem. It does not assert that the optimal policy is unique.

The array `mdp_chi.d` represents the complete deterministic optimal policy  $\pi^* = (\pi_t^*)_{t=0}^{n-1}$  without storing a separate probability array. Using the manuscript’s Iverson notation, its interpretation is

$$\pi_t^*(s, a) := \llbracket a = d_t^*(s) \rrbracket. \quad (173)$$

Thus, the stored decision-function value receives probability one, and the other action receives probability zero. An agent can execute this policy by looking up the decision-function value directly.

The subsequent `np.max` operation implements

$$\tilde{v}_t(s) := \max_{a \in A_\chi} q_t(s, a). \quad (174)$$

The code then exchanges the references `v_tt` and `v_t`. This makes the just-computed values available as continuation values for the preceding stage, while allowing the other buffer to be overwritten. Exchanging these references does not copy the arrays. Only two stages of state values are held in memory, although the decision-function values are retained for every stage.

After the final iteration, `mdp_chi.v` contains the optimal values at stage zero, and `mdp_chi.q` contains the action values for every stage. Theorem 9 identifies the recursively constructed values with the optimal values. Because the initial distribution of the reduced model places all mass on row zero, `mdp_chi.v[0]` is the initial expected optimal return. The function allocates its output arrays on each call and recomputes the rewards and policy for the supplied parameters. It does not execute the policy or simulate the original stimulus-generating environment.

### Mapping a reduced state to its array row

To evaluate the optimal decision function, A1 first extracts the reduced state  $\chi(s_t) = (s_t^3, s_t^4)$  from the full observed state. It then determines the row of this pair in the reduced state array. The builder places the initial state in row zero, followed by all states with previous task 1 and then all states with previous task 2. Within each task group, the imbalance increases from  $-n$  to  $n$ . Each group therefore contains  $2n + 1$  rows. Python uses zero-based array indices.

For a regular state, the corresponding row index is

$$k := 1 + (s_t^3 - 1)(2n + 1) + (s_t^4 + n). \quad (175)$$

The three terms have distinct roles. The leading 1 skips the initial-state row. The term  $(s_t^3 - 1)(2n + 1)$  selects the task group: it is zero for previous task 1 and  $2n + 1$  for previous task 2. Finally,  $s_t^4 + n$  converts the imbalance into an offset from zero to  $2n$  within the selected group. Consequently, the regular states with previous task 1 occupy rows 1 through  $2n + 1$ , and those with previous task 2 occupy rows  $2n + 2$  through  $4n + 2$ .

In the code, `s3` and `s4` contain  $s_t^3$  and  $s_t^4$ , `self.nb` is the cardinality  $2n + 1$  of the reduced model's imbalance set, and `self.task.n` is the full model's horizon  $n$ . The full and reduced models use the same horizon and imbalance bound. This gives the assignment

```
k = 1 + (s3 - 1) * self.nb + s4 + self.task.n # reduced-state row index
```

For example, with  $n = 2$ , the imbalance set has five elements. The reduced state  $(1, 0)$  is stored in row  $1 + 0 + 2 = 3$ , while  $(2, 0)$  is stored in row  $1 + 5 + 2 = 8$ . The initial state  $(\emptyset, 0)$  is handled separately by setting  $k = 0$ , since the regular-state formula does not apply when no previous task exists.

This calculation is an arithmetic lookup into the existing state ordering, not an additional state transition. It avoids searching the state array on each action. A1 then reads `self. [t, k]`, the value of the optimal decision function at the current stage and reduced state, before applying the lifting function  $\lambda$  to obtain the full-MDP action. Here,  $k$  indexes the current reduced state. The successor-index table used during backward induction applies the same row-numbering convention to successor states.

## S4 Successive summation

**Theorem 11** (Successive summation of products of normalized functions). *For  $i = 1, \dots, n$ , let  $D_i$  denote nonempty finite sets and let*

$$f_i : D_1 \times \dots \times D_i \rightarrow \mathbb{R}_{\geq 0}, x_{1:i} \mapsto f_i(x_{1:i}) \quad (176)$$

*denote functions satisfying*

$$\sum_{x_i \in D_i} f_i(x_{1:i}) = 1 \quad (177)$$

*for every fixed  $x_{1:i-1}$ . Then,*

$$\sum_{x_{1:n}} \left( \prod_{i=1}^n f_i(x_{1:i}) \right) = 1. \quad (178)$$

*More generally, for  $t = 1, \dots, n-1$  and every fixed  $x_{1:t}$ ,*

$$\sum_{x_{t+1:n}} \left( \prod_{i=1}^n f_i(x_{1:i}) \right) = \prod_{i=1}^t f_i(x_{1:i}). \quad (179)$$

*Proof.* The assertions follow from the linearity of finite sums and the assumed normalization of the functions. We first establish the first assertion by induction.

*Base case.* Let  $n = 2$ . Then,

$$\begin{aligned} & \sum_{x_{1:2}} \left( \prod_{i=1}^2 f_i(x_{1:i}) \right) \\ &= \sum_{x_1} \sum_{x_2} f_1(x_1) f_2(x_1, x_2) \\ &= \sum_{x_1} f_1(x_1) \left( \sum_{x_2} f_2(x_1, x_2) \right) \\ &= \sum_{x_1} f_1(x_1) \\ &= 1. \end{aligned}$$

For  $n = 1$ , the assertion is the normalization assumption itself.

*Induction step.* Assume that the assertion holds for  $n-1$ . Then,

$$\begin{aligned} & \sum_{x_{1:n}} \left( \prod_{i=1}^n f_i(x_{1:i}) \right) \\ &= \sum_{x_{1:n-1}} \sum_{x_n} \left( \left( \prod_{i=1}^{n-1} f_i(x_{1:i}) \right) f_n(x_{1:n}) \right) \\ &= \sum_{x_{1:n-1}} \left( \left( \prod_{i=1}^{n-1} f_i(x_{1:i}) \right) \left( \sum_{x_n} f_n(x_{1:n}) \right) \right) \\ &= \sum_{x_{1:n-1}} \left( \prod_{i=1}^{n-1} f_i(x_{1:i}) \right) \end{aligned}$$

$$= 1,$$

where the penultimate equality follows from normalization and the final equality follows from the induction assumption.

For the second assertion, fix  $x_{1:t}$ . Applying the first assertion to the remaining  $n - t$  arguments gives

$$\begin{aligned} & \sum_{x_{t+1:n}} \left( \prod_{i=1}^n f_i(x_{1:i}) \right) \\ &= \sum_{x_{t+1:n}} \left( \left( \prod_{i=1}^t f_i(x_{1:i}) \right) \left( \prod_{i=t+1}^n f_i(x_{1:i}) \right) \right) \\ &= \left( \prod_{i=1}^t f_i(x_{1:i}) \right) \left( \sum_{x_{t+1:n}} \left( \prod_{i=t+1}^n f_i(x_{1:i}) \right) \right) \\ &= \prod_{i=1}^t f_i(x_{1:i}). \end{aligned}$$

□

Unlike the univariate functions in Theorem S.1 of Ostwald et al. (2026), each  $f_i$  may depend on all preceding arguments, not just  $x_i$ . For the ABM, these functions are the conditional probability factors

$$f_i(x_{1:i}) = p(x_i | x_{1:i-1}), \tag{180}$$

with any arguments irrelevant to a particular factor omitted from its probability notation.